STSI4105
Edisi 2

Panduan Praktikum Transaksi dan Basis Data Terdistribusi

Penulis:
Andri Suryadi, S.Kom., M.Kom.

Penelaah:
Unggul Utan Sufandi, S.Kom., M.Si.

Kata Pengantar

Mata kuliah basis data sangat penting bagi mahasiswa yang sedang mengejar gelar di bidang sistem informasi, komputer, atau teknologi informasi. Matakuliah ini memberikan pengetahuan dan pengalaman secara praktis kepada mahasiswa agar mampu mengorganisir data yang tersimpan secara elektronik pada sebuah sistem komputer sehingga menghasilkan informasi yang berguna

Panduan ini dirancang khusus untuk para mahasiswa dengan tujuan memberikan pemahaman mendalam dan pengalaman praktis dalam bidang basis data, yang merupakan salah satu faktor penting dalam dunia teknologi informasi saat ini.

Panduan ini menggabungkan materi-materi esensial, petunjuk praktik langkah demi langkah, serta skenario tugas praktis yang akan menguji kemampuan Anda dalam merancang, mengonfigurasi, dan mengatasi masalah terkait basis data. Setiap praktikum didesain untuk meningkatkan pemahaman Anda tentang konsep-konsep kunci dalam basis data, sekaligus memberikan keterampilan yang dapat Anda terapkan di dunia nyata.

Selamat belajar dan semoga panduan praktikum mata kuliah Basis Data ini memberikan kontribusi yang signifikan bagi perkembangan pengetahuan dan keterampilan Anda, para mahasiswa, dalam dunia basis data.

Daftar Isi

Kata Pengantar

Daftar isi

BAB1 Pendahuluan

Latar Belakang

Tujuan

Ruang Lingkup

BAB2 Prosedur Pelaksanaan Praktik/Praktikum

Tahapan Praktik/Praktikum

- Pengantar Transaksi dan Basis Data Terdistribusi
- Pelaksanaan
- Pelaporan dan Penilaian

BAB 1
Pendahuluan

BAB 1

Pendahuluan

A. LATAR BELAKANG

Halo para mahasiswa yang sedang mengikuti mata kuliah Basis Data di Program Studi Sistem Informasi Universitas Terbuka! Panduan praktikum ini untuk mendampingi kegiatan belajar mahasiswa dalam memahami konsep-konsep tentang basis data.

Panduan praktikum ini dirancang khusus dalam memperoleh pemahaman yang mendalam tentang dunia basis data. Di dalam panduan ini, mahasiswa akan menemukan petunjuk langkah demi langkah, penjelasan teknis yang mudah dipahami, dan contoh kasus nyata yang akan membantu mahasiswa memahami bagaimana basis data diimplementasikan dalam kehidupan sehari-hari.

Melalui panduan ini harapannya akan memberikan inspirasi, motivasi, dan tuntunan dalam mempelajari basis data. Pada akhirnya, tujuan kami adalah memberikan mahasiswa pengetahuan dan keterampilan yang relevan, sehingga membuka peluang karir yang menarik di dunia teknologi informasi.

B. TUJUAN

Panduan praktikum ini disusun dengan tujuan utama untuk membantu mahasiswa dalam mencapai beberapa target penting dalam mata kuliah Basis Data. Berikut adalah beberapa tujuan yang ingin kami capai melalui panduan ini:

1. Pemahaman Konsep Dasar: Panduan praktikum ini bertujuan untuk memberikan pemahaman yang kuat tentang konsep dasar basis data.
2. Pengalaman Praktis: Kami ingin memberikan pengalaman praktis kepada mahasiswa agar mereka dapat menerapkan konsep-konsep yang telah dipelajari ke dalam situasi nyata. Dengan melakukan praktikum dan tugas yang terkait, mahasiswa akan memperoleh keterampilan praktis yang berguna dalam merancang, mengkonfigurasi, dan mengelola basis data.
3. Penerapan Pengetahuan: Panduan ini bertujuan untuk membantu mahasiswa dalam mengaplikasikan pengetahuan teoritis yang telah diperoleh dari mata kuliah ke dalam lingkungan praktis.

Melalui praktikum, mahasiswa akan dapat menghubungkan teori dengan situasi nyata dalam pengaturan basis data, serta menghadapi tantangan dan menyelesaikan masalah yang mungkin timbul.

4. Pengembangan Kemampuan Problem Solving: Tujuan lain dari panduan ini adalah mengembangkan kemampuan mahasiswa dalam pemecahan masalah. Melalui praktikum yang menantang, mahasiswa akan dihadapkan pada skenario yang memerlukan analisis, pemecahan masalah, dan pengambilan keputusan yang tepat. Hal ini akan membantu mahasiswa dalam mengasah kemampuan kritis dan pemecahan masalah yang sangat diperlukan terkait basis data..
5. Persiapan Karir: Akhirnya, panduan ini ditujukan untuk mempersiapkan mahasiswa untuk karir di bidang basis data. Dengan memahami konsep-konsep yang relevan dan menguasai keterampilan praktis yang diperlukan, mahasiswa akan memiliki landasan yang kuat untuk memasuki industri teknologi informasi dan menjalani peran profesional dalam merancang, mengelola, dan mengamankan basis data.

Melalui pencapaian tujuan-tujuan ini, panduan praktikum ini diharapkan dapat memberikan kontribusi yang signifikan dalam perkembangan pengetahuan, keterampilan, dan persiapan karir mahasiswa dalam dunia basis data.

C. RUANG LINGKUP

Panduan praktikum ini memiliki ruang lingkup yang luas dan komprehensif untuk membantu mahasiswa dalam memahami dan menguasai konsep-konsep basis data. Berikut adalah beberapa ruang lingkup yang akan dicakup dalam panduan ini:

Praktik 1 : Pembuatan Entity Relationship Diagram (ERD)

Praktik 2 : SQL dan Data Manipulation Language (DML)

Praktik 3 : VirtualBox, Transaksi dan Basis Data Terdistribusi

BAB 2

Prosedur Pelaksanaan Praktik/Praktikum

A. PENGANTAR TRANSAKSI DAN BASIS DATA TERDISTRIBUSI

Transaksi

Seperti yang telah dijelaskan pada Modul 7, transaksi berfungsi untuk menjaga konsistensi data. Supaya terjamin konsistensi data tersebut, terdapat properti-properti dalam transaksi yaitu ACID (*Atomicity, Consistency, Isolation, Durability*). *Atomicity* merupakan transaksi yang dilakukan seluruhnya atau tidak sama sekali, *consistency* merupakan data di dalam basis data harus tetap bersifat konsisten, *isolation* merupakan transaksi yang terisolasi dan tidak boleh diganggu oleh transaksi lain, dan *durability* merupakan daya tahan data yang harus terjamin. Selain itu, di dalam sebuah transaksi terdapat 5 perintah yang memungkinkan yaitu *active, partially commited, failed, abort*, dan *commit*. Kelima perintah tersebut telah dibahas pada Modul 7.

Dalam praktikum Transaksi ini, Anda akan mencoba beberapa perintah dalam transaksi yaitu *commit*, *rollback*, *checkpoint*, dan *trigger*. Perintah-perintah tersebut akan dicoba menggunakan basis data MySQL yang ada pada aplikasi XAMPP. Terkait dengan MySQL dan XAMPP, silahkan buka kembali materi yang ada di Modul 6.

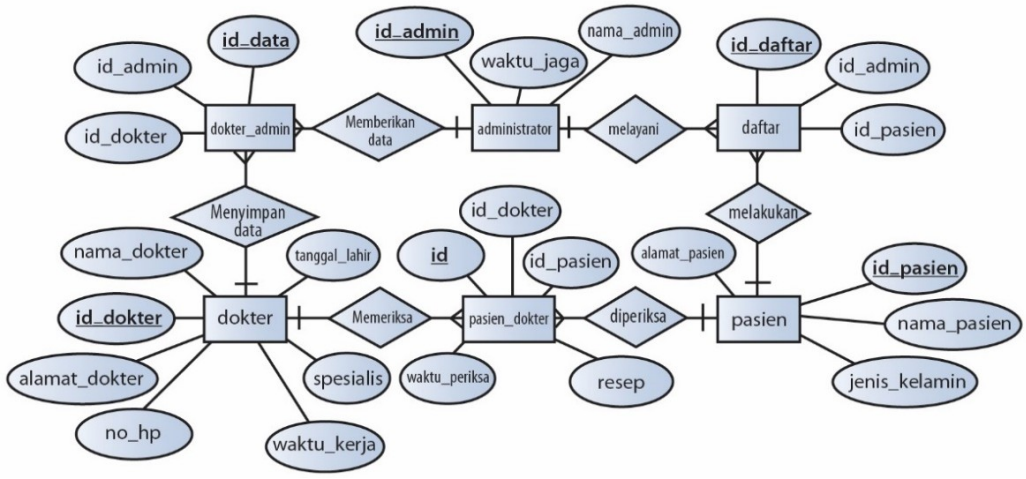
Basis Data Terdistribusi

Pada kegiatan praktikum Basis Data Terdistribusi ini, Anda akan mencoba praktikum memasang aplikasi xampp pada komputer 2 (kemudian pratikum basis data terdistribusi dalam hal replikasi basis data dari komputer 1 (Windows 10) ke komputer 2 (Ubu ntu). Namun, sebelum memulai praktikum kegiatan praktikum 3 ini pastikan komputer 1 dan komputer 2 telah terhubung dengan baik.

B. PELAKSANAAN

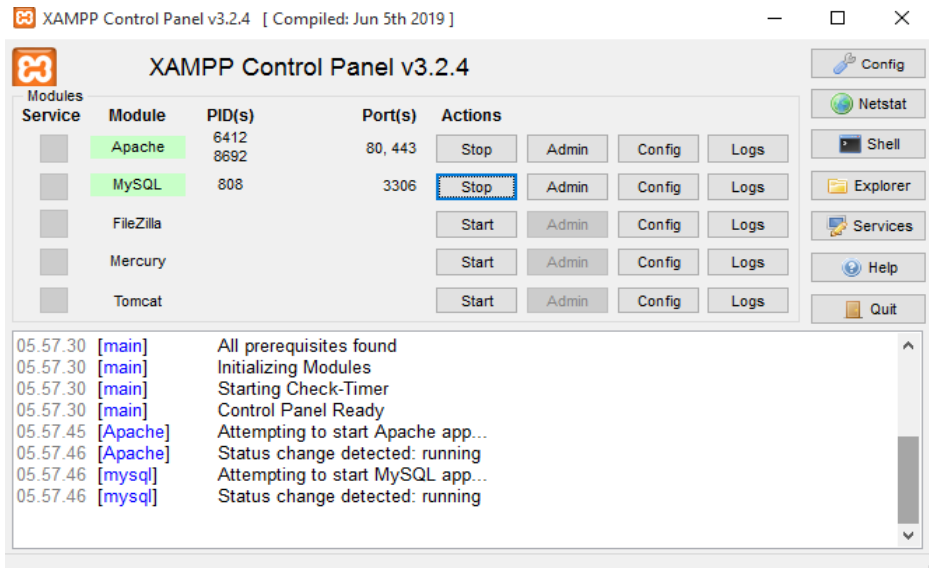
B.1 PELAKSANAAN PRAKTIKUM TRANSAKSI

Basis data yang akan digunakan adalah basis data yang telah dibuat pada praktikum Modul 6 yaitu basis data rumah sakit. Adapun ERD (*Entity Realationship* Diagram) dapat Anda lihat pada Gambar 1 berikut ini.



Gambar 1
ERD Rumah Sakit

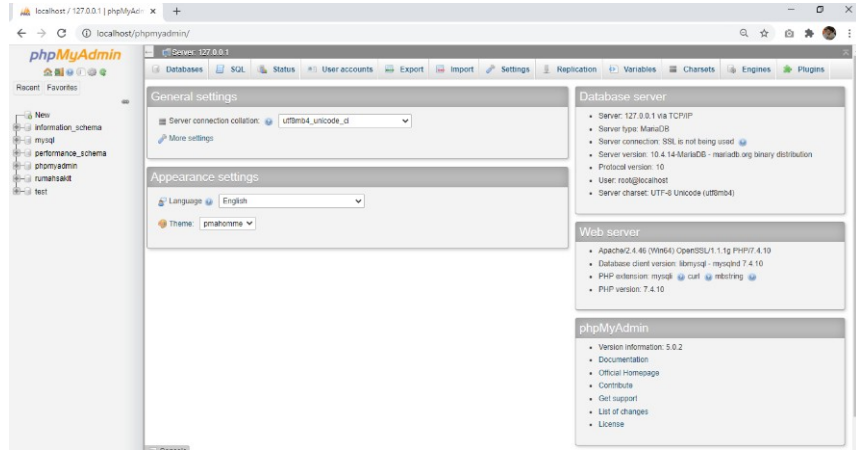
Perlu diingat, dalam praktikum transaksi ini diasumsikan aplikasi xampp telah terpasang dan halaman pengaturan basis data akan menggunakan halaman phpmyadmin. Namun, untuk dapat masuk ke halaman tersebut, terlebih dahulu aktifkan apache dan MySQL pada aplikasi XAMPP yang dapat dilihat pada Gambar 2 berikut.



Gambar 2
Control XAMPP

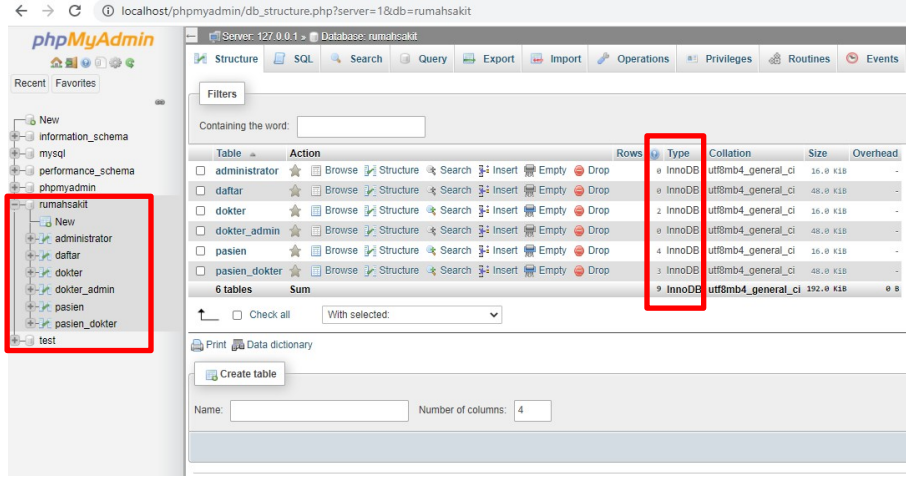
Berikutnya buka web browser yang ada di komputer Anda seperti mozilla firefox, google chrome dan sebagainya. Kemudian masukan alamat URL :

`http://localhost/phpmyadmin/` sehingga akan tampil seperti yang dapat Anda lihat pada Gambar 3 berikut.



Gambar 3
Halaman Localhost

Selanjutnya, untuk dapat melakukan transaksi, basis data MySQL harus dapat mendukung salah satu *storage engine* yaitu InnoDB atau BDB (BerkeleyDB). Cara memeriksa apakah MySQL Anda mendukung salah satu *storage engine* Anda `klik basis data rumah sakit` kemudian lihat apakah tabel-tabel yang ada pada basis data tersebut sudah InnoDB seperti yang dapat Anda lihat pada Gambar 4 berikut.



Gambar 4
status InnoDB pada Basis Data Rumah Sakit

Jika type tabel sudah InnoDB maka traksaksi dapat berjalan pada basis data MySQL tersebut. Selanjutnya, untuk membuat sebuah transaksi dalam halaman phpmyadmin, Anda dapat membuatnya dengan cara memasukan perintah pada menu

SQL yang diawali perintah **begin** dan diakhiri dengan **end** kemudian disimpan pada *store prosedure*. *Store prosedure* adalah objek *routine* yang disimpan pada basis data MySQL dan dapat menggantikan sekumpulan perintah yang sering digunakan. intinya *store prosedure* ini merupakan kumpulan-kumpulan perintah yang telah terprogram dan siap dipanggil untuk melakukan perintah oleh aplikasi.

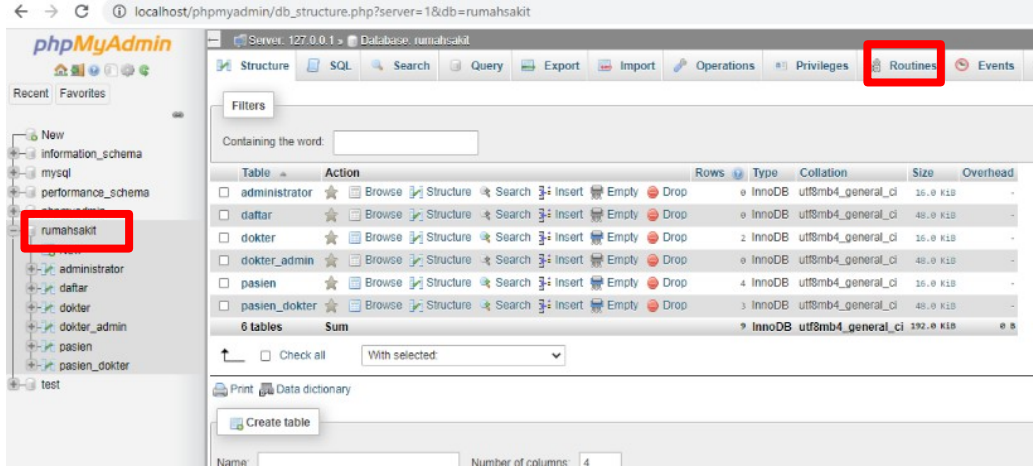
1. *Commit*

Commit adalah sebuah perintah yang menandakan bahwa transaksi telah selesai dilaksanakan dan terjadi perubahan data. Sebagai contoh akan membuat *store prosedure* dengan sebuah perintah *commit* dengan memasukan data pada tabel pasien. Misalnya, akan memasukan data pasien tanpa perintah *commit* terlebih dahulu dengan atribut id_pasien=5, nama_pasien=Tito, alamat_pasien=Jl Pakis haji no 25, dan jenis kelamin=1. Maka perintah yang akan dimasukan pada store prosedure adalah sebagai berikut.

```
1  BEGIN
2  START TRANSACTION;
3  INSERT INTO `pasien` (`id_pasien`, `nama_pasien`,
4  `alamat_pasien`, `jenis_kelamin`)
5  VALUES (6, 'Tito', 'Jl.Pakis Haji no 23', '1');
6  SELECT * FROM pasien;
7  END
```

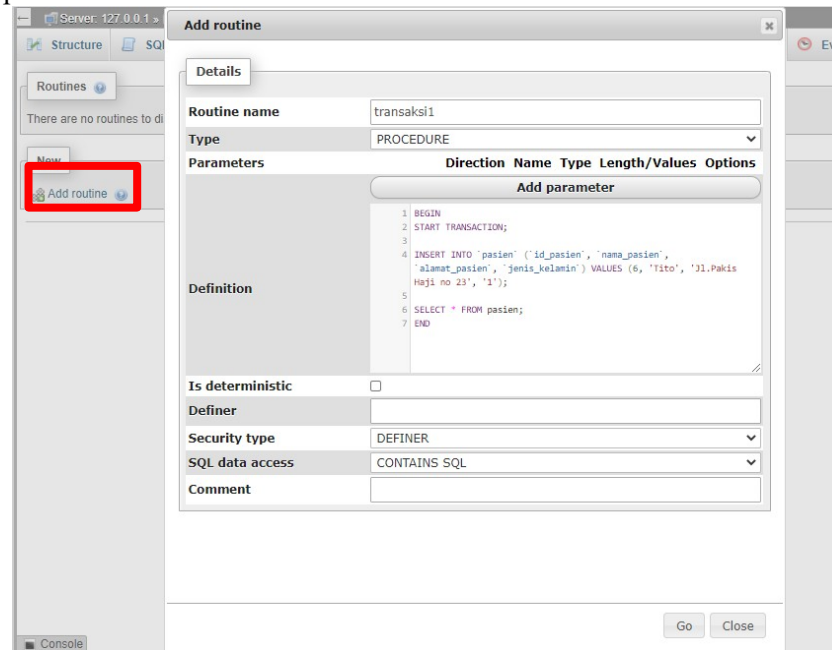
Gambar 5
Perintah *Commit*

Perintah tersebut tanpa *commit* dan akan dimasukan ke dalam *store procedure* yang ada di halaman phpmyadmin. Dengan cara `Pilih basis data rumah sakit` → `routines`. Lihat Gambar 6 berikut ini.



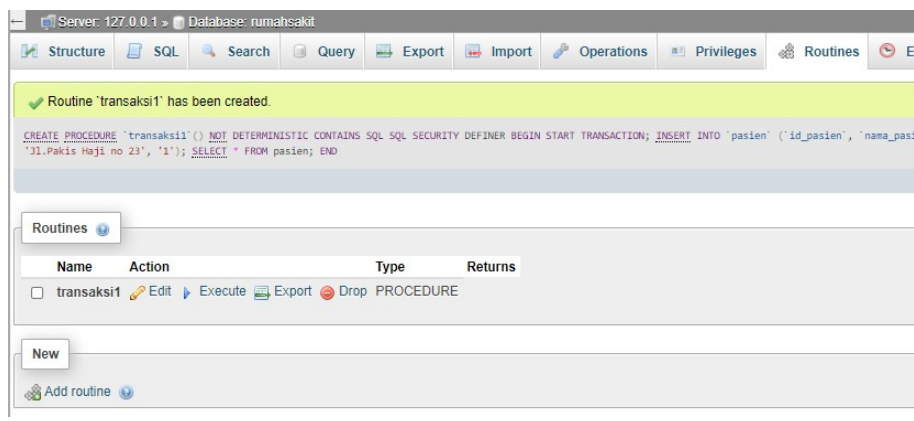
Gambar 6
Membuat *Store Procedure*

Kemudian pilih `add routine` → `isi data` → `Go` dan masukan perintah di atas seperti pada Gambar 7 berikut ini.



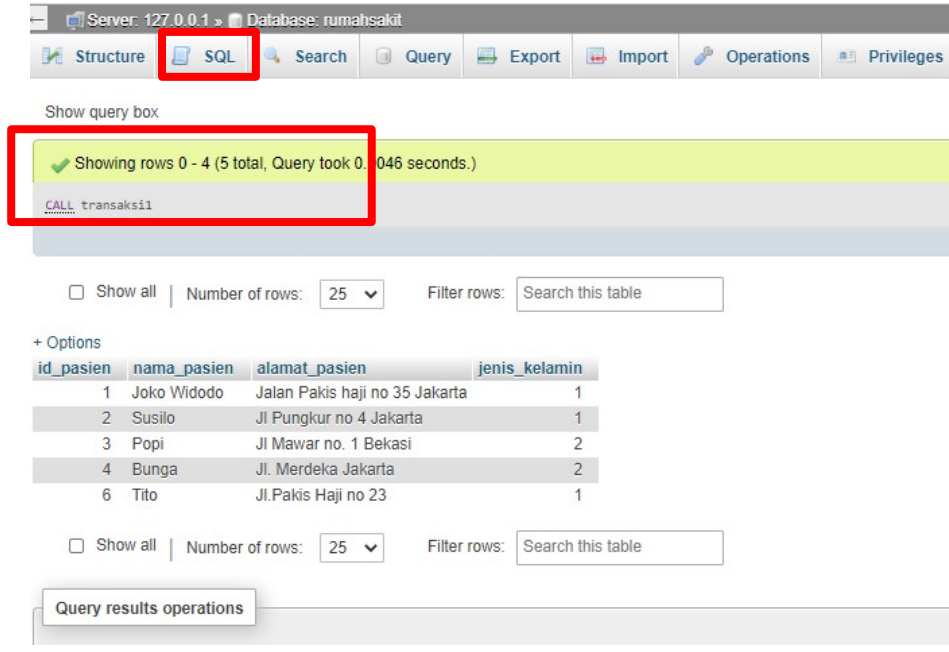
Gambar 7
Membuat Transaksi1

Kemudian akan muncul sebuah dialog dan isi seperti Gambar 7. nama *routine name* isi dengan transaksi1 (penamaan tidak dipisah). Jadi Anda akan membuat sebuah *store procedure* dengan nama transaksi1 berisi perintah yang ada pada *definition* yaitu memasukan data pada tabel pasien. Jika berhasil maka akan bertambah 1 buah *store procedure* pada *routine* seperti pada Gambar 8 berikut ini.



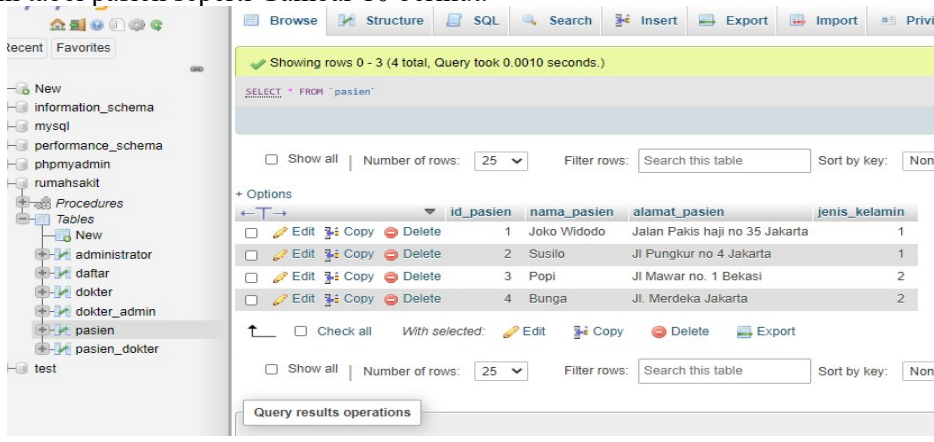
Gambar 8
Transaksi1

Selanjutnya adalah mencoba menjalankan transaksi1 dengan perintah CALL pada SQL. Maka langkah untuk menjalankan transaksi1 Anda `pilih menu SQL` → `masukan perintah "CALL transaksi1;"` seperti yang dapat Anda lihat pada Gambar 9.



Gambar 9
Call Transaksi1

Gambar 9 adalah hasil dari menjalankan transaksi1 melalui menu SQL. Hasilnya dapat Anda lihat yaitu data dengan id_pasien 6 telah tampil ke dalam layar. Namun, Anda perlu ingat bahwa transaksi1 tersebut tanpa *commit* sehingga hasilnya tidak akan masuk ke dalam tabel pasien. Untuk memeriksa data tersebut masuk atau tidak silahkan pilih tabel pasien seperti Gambar 10 berikut.



Gambar 10
Memeriksa Tabel Pasien

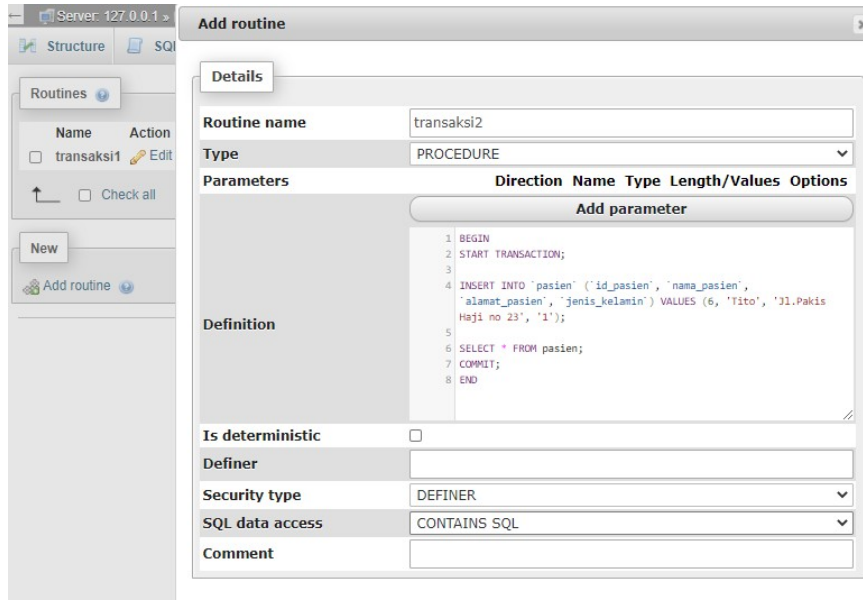
Perhatikan Gambar 10 bahwa dalam tabel pasien tidak ada id_pasien 6. Artinya apapun perintah yang disimpan pada sebuah transaksi jika belum *commit* maka data tidak akan masuk ke dalam tabel basis data. Supaya data tersebut masuk ke dalam tabel pasien, maka perintah dalam transaksi 1 harus dimodifikasi dengan menambahkan *commit*.

Sebagai contoh Anda buat dengan nama *store procedure* baru misal transaksi2. Dalam transaksi2 merupakan modifikasi perintah dengan menambahkan *commit* seperti berikut.

```
1  BEGIN
2  START TRANSACTION;
3  INSERT INTO `pasien` (`id_pasien`, `nama_pasien`,
4  `alamat_pasien`, `jenis_kelamin`)
5  VALUES (6, 'Tito', 'Jl.Pakis Haji no 23', '1');
6  SELECT * FROM pasien;
7  COMMIT;
8  END
```

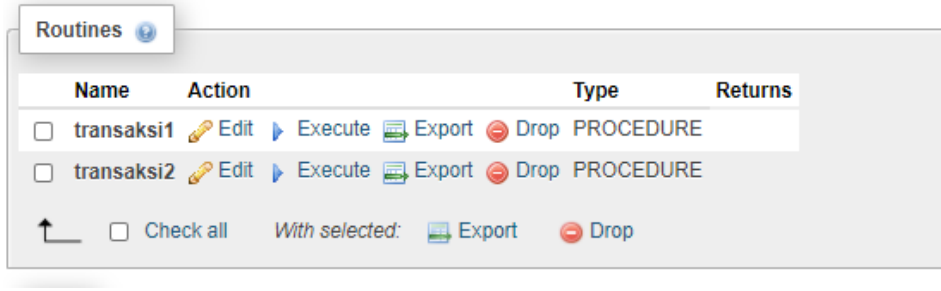
Gambar 11
Modifikasi *Commit*

Pada halaman *routines* dapat Anda lihat pada Gambar 9.12 sebagai berikut.



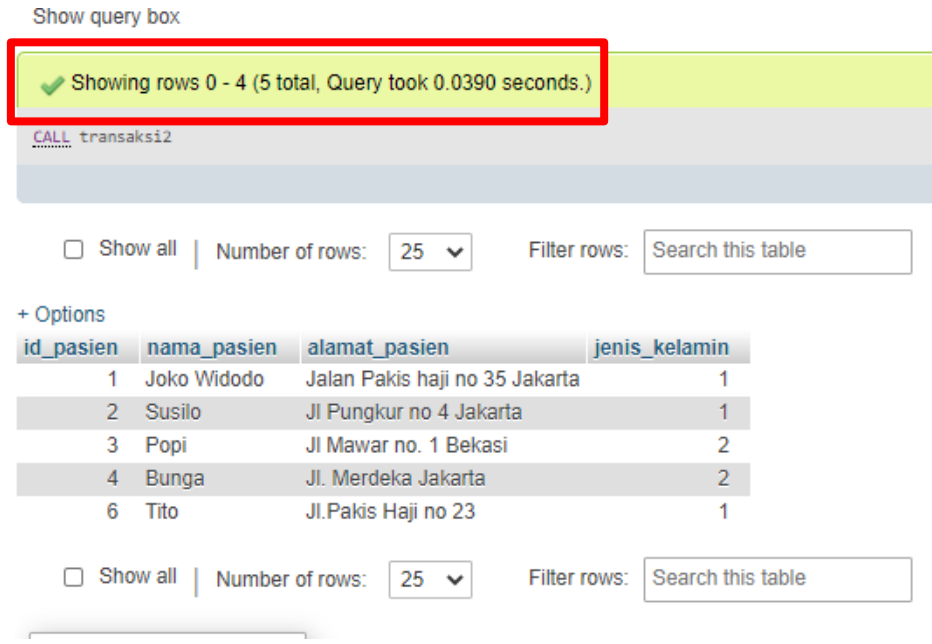
Gambar 12
Menambah Transaksi2

Maka dapat Anda lihat sekarang terdapat 2 routines. Lihat Gambar 13 berikut.



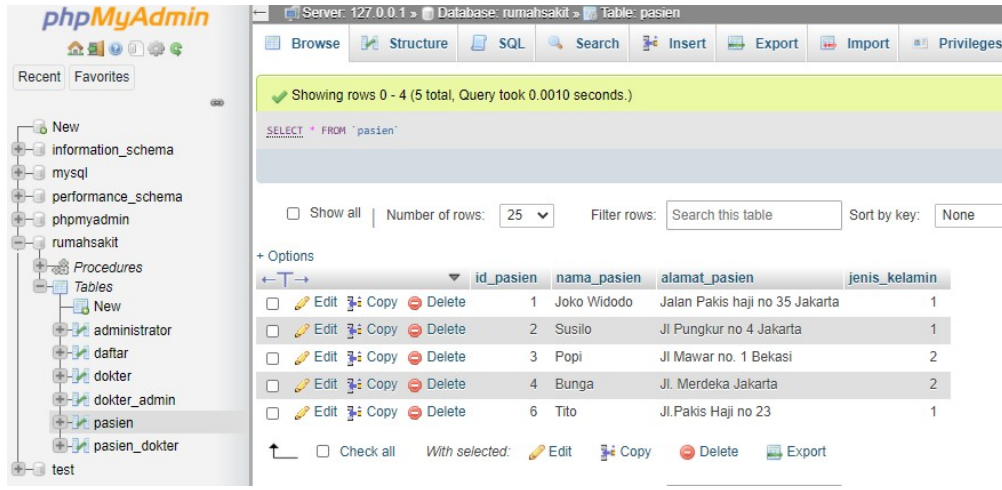
Gambar 13
Routines

Untuk menjalankan transaksi 2, seperti yang telah dilakukan pada transaksi 1 yaitu dengan menjalankan perintah CALL pada SQL. Seperti pada Gambar 14 berikut ini.



Gambar 14
Menjalankan Transaksi 2

Dapat Anda lihat pada Gambar 14 bahwa id pasien 6 telah tampil di layar. Untuk memastikan bawah perintah *commit* berjalan dengan baik maka silahkan lihat tabel pasien seperti pada Gambar 15 berikut ini.



Gambar 15
Tabel Pasien Setelah Transaksi 2

Gambar 15 menunjukkan bahwa data pasien dengan id pasien 6 telah masuk ke dalam tabel pasien. Artinya, perintah *commit* telah berjalan dengan baik. Kesimpulan dari *commit* adalah jika sebuah perintah SQL baik memasukkan, merubah atau menghapus data telah dijalankan namun belum sampai *commit* maka perintah tersebut tidak akan merubah isi data tabel.

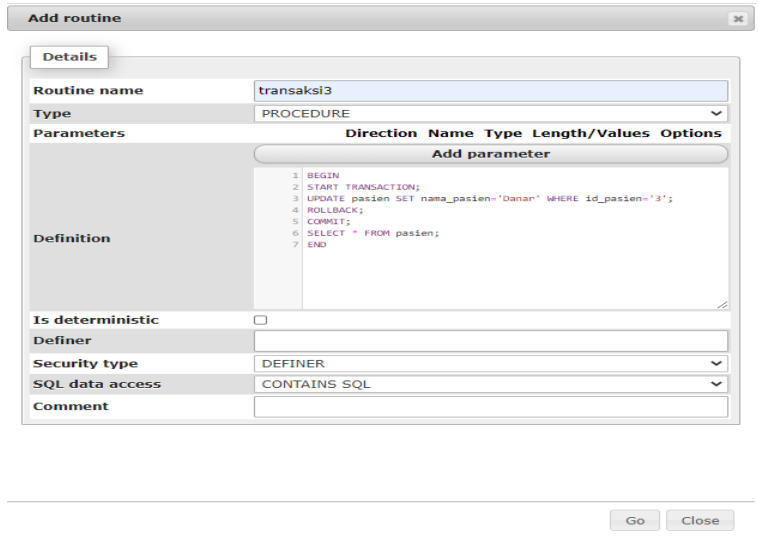
2. Rollback

Pernyataan *rollback* digunakan untuk menggugurkan perintah yang sedang berjalan. Hal tersebut berguna jika ada suatu hal yang tidak diinginkan terjadi pada sistem. Namun, perlu diingatkan bahwa pernyataan *rollback* akan berpengaruh jika perintah yang sedang berjalan belum *commit*. Misalnya Anda akan menjalankan perintah *update* pada data di tabel pasien dengan melakukan perubahan nama pasien Popi menjadi Danar dengan id pasien = 3. Namun, dengan pernyataan *rollback* maka perubahan tersebut tidak terjadi. Berikut adalah contoh pernyataan *rollback* pada transaksi3.

```
1  BEGIN
2  START TRANSACTION;
3  UPDATE pasien
4  SET nama_pasien='Danar'
5  WHERE id_pasien='3';
6  ROLLBACK;
7  COMMIT;
8  SELECT * FROM pasien;
9  END
```

Gambar 16
Perintah *Rollback*

Pertanyaan di atas merupakan pernyataan melakukan perubahan nama pasien menjadi danar pada id pasien 3. Namun, dengan adanya *rollback* maka perubahan tersebut tidak terjadi dan setelah *commit* data ditampilkan. Jika diimplementasikan pada halaman phpmyadmin maka perhatikan Gambar 17 berikut ini.



Gambar 17
Pernyataan *Rollback*

Untuk melihat hasil yang terjadi maka jalankan dengan cara memanggil transaksi 3 melalui perintah **CALL transaksi3**. Hasil yang terjadi pada pertanyaan di atas adalah sebagai berikut.



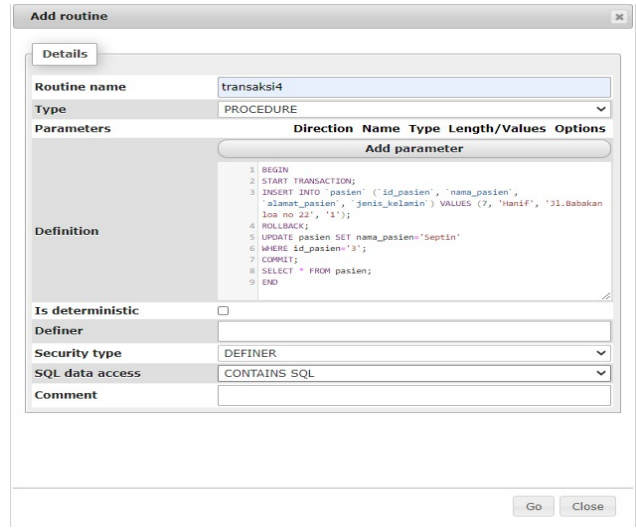
Gambar 18
Hasil pada Tabel Pasien Setelah *Rollback*

Pada Gambar 18 tidak terjadi perubahan data karena telah dilakukan *rollback*. Jika ingin ada perubahan maka transaksi 3 tersebut tidak memakai *rollback*. Untuk lebih jelas terkait dengan *rollback*, maka perhatikan kasus pada transaksi 4 berikut. Misalnya akan dilakukan perintah *insert* data baru pada tabel pasien dan pada id pasien 3 akan dilakukan perubahan data (*update*) nama pasien menjadi Septin. Namun, setelah perintah *insert* dilakukan, perintah selanjutnya adalah *rollback* dan setelah itu dilakukan perubahan pada nama pasien menjadi Septin. Berikut pernyataan 3 yang terjadi.

```
BEGIN
START TRANSACTION;
INSERT INTO `pasien` (`id_pasien`, `nama_pasien`,
`alamat_pasien`, `jenis_kelamin`)
VALUES (7, 'Hanif', 'Jl.Babakan loa no 22', '1');
ROLLBACK;
UPDATE pasien
SET nama_pasien='Septin'
WHERE id_pasien='3';
COMMIT;
SELECT * FROM pasien;
END
```

Gambar 19
Modifikasi *Rollback*

Pada pernyataan di atas, terjadi 2 kali perubahan data yaitu memasukkan data baru (*insert*) dan perubahan data pasien (*update*). Namun setelah perintah *insert* pada pernyataan pertama dilanjutkan perintah *rollback*. Implementasi pada halaman phpmyadmin adalah sebagai berikut.



Gambar 20
Transaksi 4

Untuk melihat hasil dari transaksi 4 maka lakukan **CALL transaksi4**. Hasilnya dapat Anda lihat pada Gambar 21 berikut.



Gambar 21
Hasil Transaksi 4

Pada Gambar 9.21 dapat Anda lihat tidak terjadi penambahan data karena di-*rollback*. Namun pada perintah *update* terjadi perubahan data id pasien 3 menjadi Septin.

3. *Check Point*

Check point merupakan tempat titik di mana proses terakhir disimpan. Seperti yang telah Anda pelajari bahwa suatu perintah *rollback* adalah membatalkan atau mengembalikan suatu perintah ke kondisi di mana transaksi belum terjadi. Namun, supaya sebuah transaksi berjalan dengan optimal, maka kondisi *rollback* tidak harus dikembalikan ke kondisi awal. Akan tetapi bisa dikembalikan ke posisi tempat *check point* terakhir. Misalnya ada 3 buah data yang akan dimasukan ke dalam basis data. Perhatikan pernyataan berikut ini.

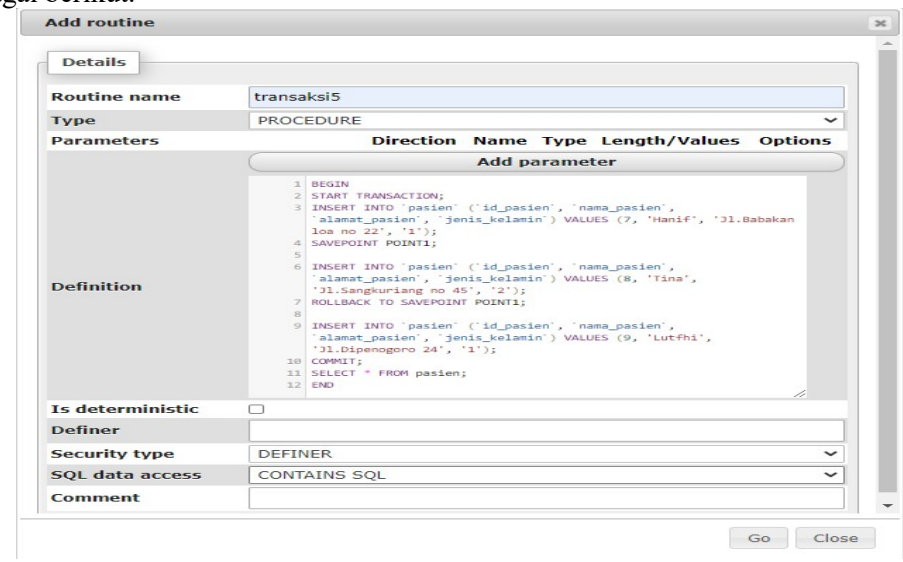
```
BEGIN
START TRANSACTION;
INSERT INTO `pasien` (`id_pasien`, `nama_pasien`, `alamat_pasien`, `jenis_kelamin`)
VALUES (7, 'Hanif', 'Jl.Babakan loa no 22', '1');
SAVEPOINT POINT1;

INSERT INTO `pasien` (`id_pasien`, `nama_pasien`, `alamat_pasien`, `jenis_kelamin`)
VALUES (8, 'Tina', 'Jl.Sangkuriang no 45', '2');
ROLLBACK TO SAVEPOINT POINT1;

INSERT INTO `pasien` (`id_pasien`, `nama_pasien`, `alamat_pasien`, `jenis_kelamin`)
VALUES (9, 'Lutfhi', 'Jl.Dipenogoro 24', '1');
COMMIT;
SELECT * FROM pasien;
END
```

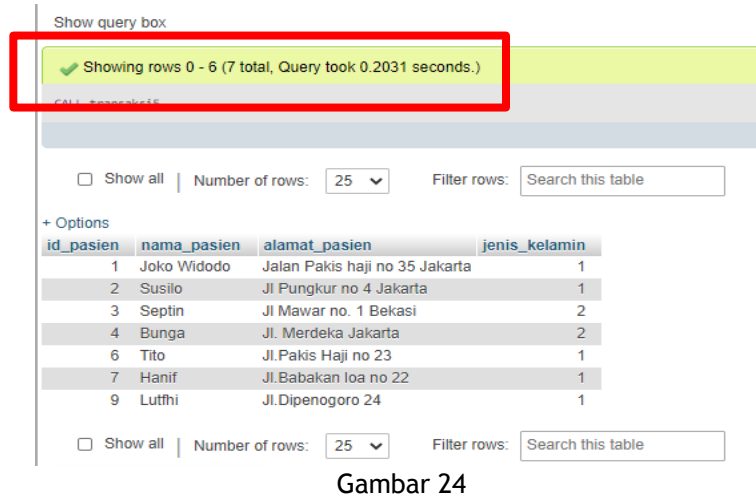
Gambar 22
Contoh *Savepoint*

Pada baris ke lima, Anda dapat lihat bahwa setelah melakukan *insert* yang pertama dilakukan *savepoint* dengan nama point1. Kemudian dilakukan *insert* yang ke dua namun di-*rollback* ke *savepoint* point1. Kemudian dilakukan kembali *insert* yang ke tiga. Implementasi pada halaman phpmyadmin dapat Anda lihat pada Gambar 23 sebagai berikut.



Gambar 23
Contoh Transaksi 5 *Savepoint*

Untuk menjalankan transaksi 5 ini dilakukan pemanggilan dengan cara **CALL transaksi5**. Maka hasil dari transaksi 5 ini dapat Anda lihat pada Gambar 24 berikut ini.



Gambar 24
Hasil dari Transaksi 5

Dari hasil transaksi 5 dapat Anda lihat data yang masuk ke dalam basis data hanya 2 yaitu id pasien 7 dan id pasien 9. Sedangkan untuk id_pasien 8 tidak masuk ke dalam basis data karena di-*rollback*.

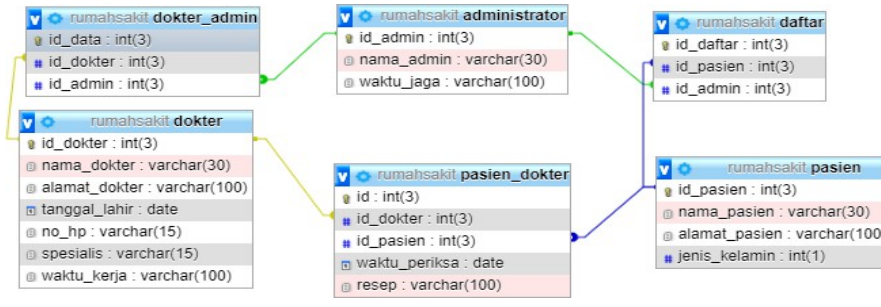
4. Trigger

Trigger merupakan sebuah perintah atau *event* yang dapat memicu perubahan sebuah basis data. *Event* yang dapat digunakan yaitu memasukkan (*insert*), perubahan (*update*), dan penghapusan (*delete*). *Event-event* tersebut ditambahkan sebuah kata kunci pemicu yang di dalam *trigger* terdapat dua buah kata kunci yaitu *before* dan *after*. *Before* dilakukan sebelum sebuah perintah (*insert, update,* atau *delete*) dilaksanakan. Sedangkan *after* dilakukan setelah sebuah perintah (*insert, update,* atau *delete*) dilaksanakan. Tabel 1 berikut adalah penjelasan tentang perintah yang ada pada *trigger*.

Tabel 1
Perintah dalam *Trigger*

No	Perintah	Keterangan
1	BEFORE INSERT	*Trigger* berjalan sebelum perintah *insert* dilaksanakan.
2	AFTER INSERT	*Trigger* berjalan setelah perintah *insert* dilaksanakan.
3	BEFORE UPDATE	*Trigger* berjalan sebelum perintah *update* dilaksanakan.
4	AFTER UPDATE	*Trigger* berjalan setelah perintah *update* dilaksanakan.
5	BEFORE DELETE	*Trigger* berjalan sebelum perintah *delete* dilaksanakan.
6	AFTER DELETE	*Trigger* berjalan setelah perintah *delete* dilaksanakan.

Untuk lebih memahami tentang *trigger*, berikutnya Anda akan mencoba dalam MySQL melalui halaman phpmyadmin. Dalam menjalankan *trigger* tentunya harus ada beberapa tabel yang telah dibuat. Perhatikan tabel-tabel yang ada pada kasus rumah sakit yang telah dibuat sebelumnya pada Modul 6. Gambar 25 adalah gambar *entity relationship diagram (*ERD*)* pada contoh kasus rumah sakit yang telah dibuat sebelumnya.



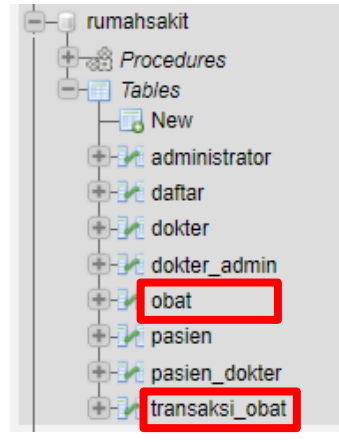
Gambar 25
Erd Kasus Rumah Sakit pada Halaman Phpmyadmin

Dari ERD kasus tersebut, Anda tambahkan 2 tabel baru yaitu tabel obat dan transaksi_obat dengan rincian sebagai berikut.

Tabel 2
Membuat Tabel Obat dan Tabel Transaksi_Obat

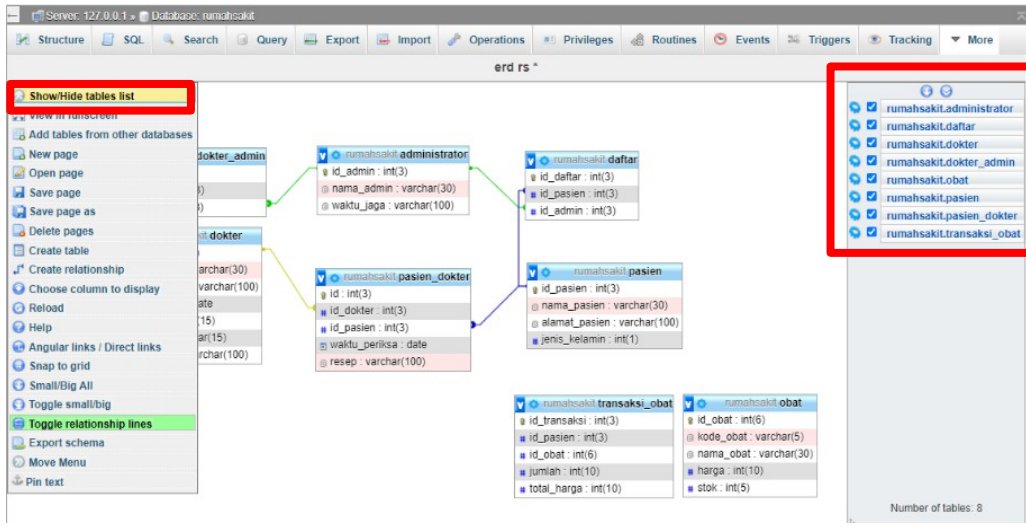
Tabel		Atribut	SQL
Tabel obat	:	id_obat (*primary key* int 3), kode_obat (varchar 5), nama_obat (var char 30), harga (int 10), stok (int 5)	CREATE TABLE obat (id_obat INT(6) AUTO_INCREMENT PRIMARY KEY, kode_obat VARCHAR(5), nama_obat VARCHAR(30), harga integer(10), stok integer(5))
Tabel transaksi_obat	:	id_transaksi (primary key int 5), id_pasien (int 3), id_obat (int 3), jumlah (int 10), total_harga (int 10)	CREATE TABLE transaksi_obat (id_transaksi int(3) AUTO_INCREMENT PRIMARY KEY, id_pasien INT(3), id_obat INT(3), jumlah int(10), total_harga int(10))

Silahkan Anda masukan kode SQL untuk membuat tabel pada menu SQL pada basis data rumah sakit sehingga hasilnya dapat anda lihat pada Gambar 9.26.



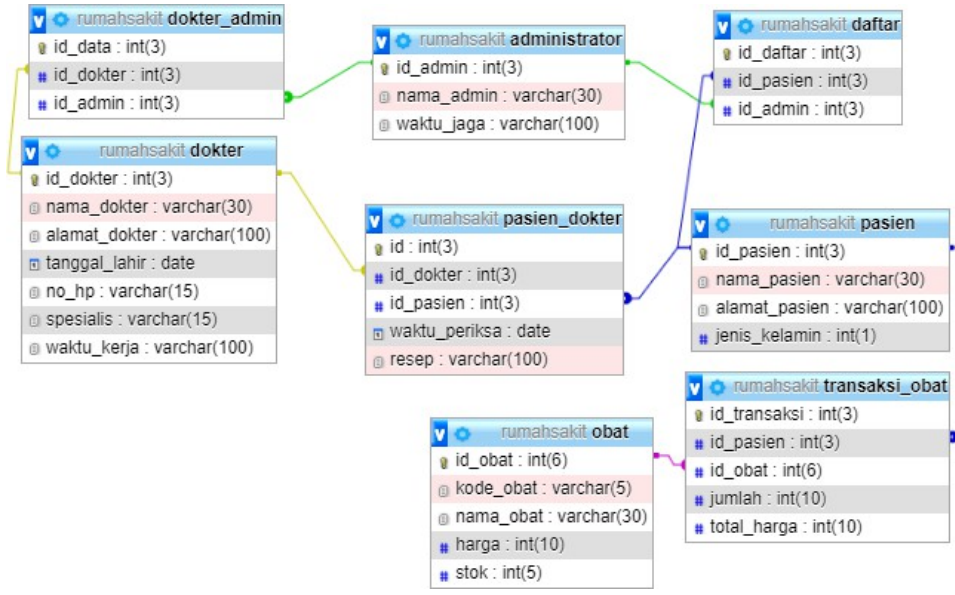
Gambar 26
Penambahan 2 Tabel pada Basis Data Rumah Sakit

Kemudian kedua tabel tersebut relasikan dengan tabel yang lainnya dengan `cara klik menu designer → kemudian klik show / hide table list → ceklist seluruh tabel` sehingga kedua tabel yang telah dibuat siap untuk direlasikan dengan tabel yang lainnya. Lihat Gambar 27 berikut ini.



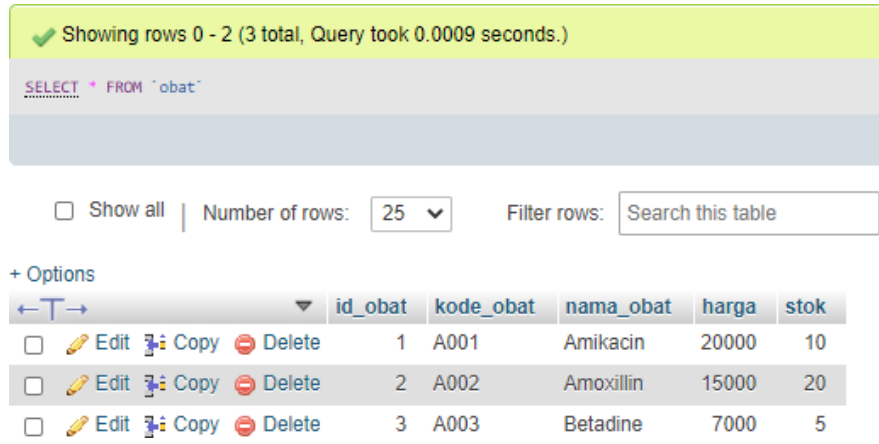
Gambar 27
Menambahkan 2 Tabel pada ERD

Selanjutnya adalah melakukan relasi dengan tabel yang telah ada yaitu antara tabel pasien, tabel transaksi, dan tabel obat. Lihat Gambar 28 berikut.



Gambar 28
Entity Relationship Diagram Kasus Rumah Sakit

Dalam Gambar 28 terlihat tabel obat dan tabel transaksi_obat telah berelasi. Maka selanjutnya dalam mencoba perintah *trigger* ini akan fokus pada ketiga tabel yaitu tabel pasien, tabel transaksi_obat, dan tabel obat. Namun, sebelum melanjutkan praktikum alangkah lebih baik diisi terlebih dahulu tabel obat minimal 3 data. Misal hasil pengisian tabel obat seperti Gambar 29 berikut ini.


Showing rows 0 - 2 (3 total, Query took 0.0009 seconds.)

```
SELECT * FROM `obat`
```

Show all | Number of rows: 25 Filter rows: Search this table

+ Options

	id_obat	kode_obat	nama_obat	harga	stok
Edit Copy Delete	1	A001	Amikacin	20000	10
Edit Copy Delete	2	A002	Amoxillin	15000	20
Edit Copy Delete	3	A003	Betadine	7000	5

Gambar 29
Contoh Data Tabel Obat

Perhatikan contoh kasus berikut ini untuk mencoba menjalankan *trigger*.

> Misalnya seorang pasien membeli obat Amoxillin sebanyak 5 buah. Maka ketika tabel transaksi_obat pada atribut jumlah dimasukan data, maka secara otomatis stok pada tabel obat akan berkurang.

Dari contoh kasus di atas dapat dibuat sebuah perintah *trigger* sebagai berikut.

```
1  DELIMITER $$
2  CREATE TRIGGER tg_jual AFTER INSERT
3   ON transaksi_obat
4   FOR EACH ROW
5   BEGIN
6      UPDATE obat SET stok = stok - new.jumlah
7      WHERE id_obat=new.id_obat;
8  END  $$
9  DELIMITER ;
```

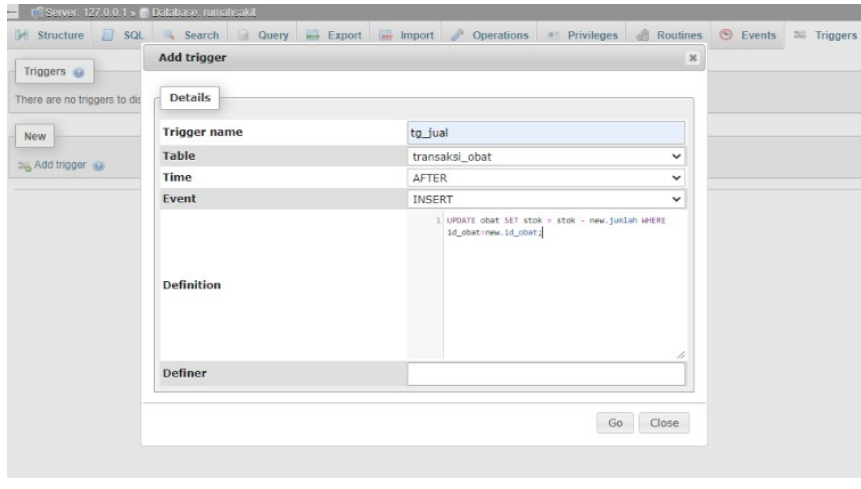
Gambar 30
Perintah *Trigger* tg_jual

Dari Gambar 9.30 dapat Anda lihat penjelasan pada Tabel 3 sebagai berikut.

Tabel 3
Penjelasan Perintah Trigger dari Gambar 30

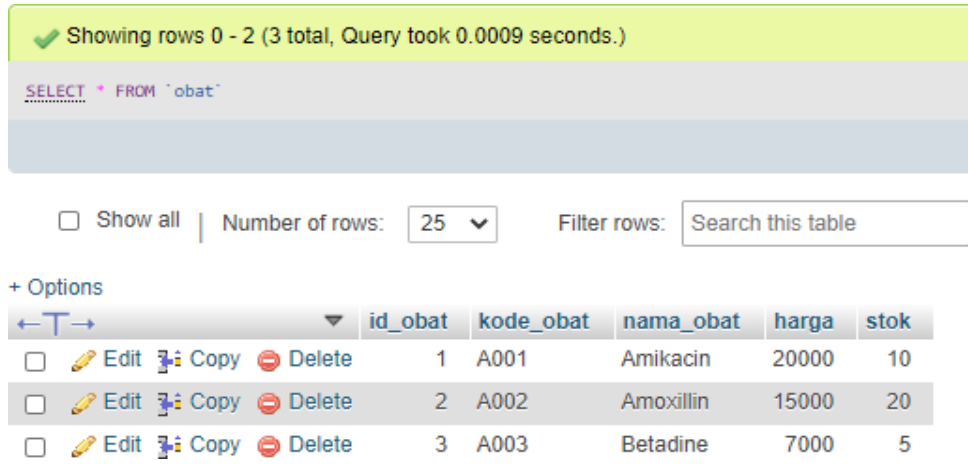
Baris	Perintah	Penjelasan
1	DELIMITER $$	Batas awal sebuah *event trigger*
2	CREATE TRIGGER tg_jual AFTER INSERT	Membuat *trigger* dengan nama tg_jual yang akan dijalankan setelah data dimasukan (*after insert*)
3	ON transaksi_obat	*Event trigger* akan bekerja pada jika ada perubahan pada tabel transaksi_obat
4	FOR EACH ROW	Untuk setiap baris pada tabel transaksi_obat
5	BEGIN	Mulai eksekusi perintah SQL
6	UPDATE obat SET stok = stok - new.jumlah	*Update* tabel obat pada atribut stok yaitu dengan mengurangi stok dengan data baru pada atribut jumlah di tabel transaksi_obat
7	WHERE id_obat=new.id_obat;	kondisi perubahan yaitu pada id_obat yang baru diinputkan pada tabel transaksi_obat dengan id_obat pada tabel obat
8	END $$	Akhir eksekusi perintah SQL
9	DELIMITER ;	Batas akhir sebuah *event trigger*

Jika diterapkan pada halaman phpmyadmin, maka langkah yang dilakukan adalah dengan cara memasukan langsung kode pada Gambar 30 atau dengan langkah `klik menu triggers → Add trigger → isi data seperti pada Gambar 31 berikut → Go.`



Gambar 31
Membuat Trigger tg_jual melalui Menu Triggers

Untuk mencoba menjalankan sebuah *trigger* ini, Anda perhatikan terlebih dahulu tabel obat pada Gambar 32.



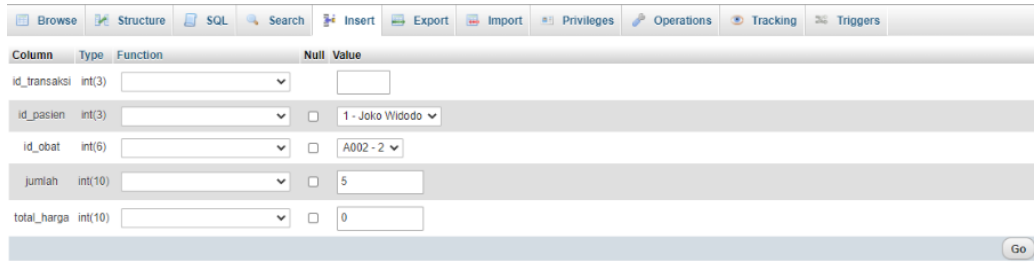
Gambar 32
Kondisi Tabel Obat

Kemudian Anda coba masukan sebuah data pada tabel transaksi obat. Misalnya dengan SQL sebagai berikut.

```
1  INSERT INTO `transaksi_obat`
2  (`id_transaksi`, `id_pasien`, `id_obat`, `jumlah`, `total_harga`)
3  VALUES (NULL, '1', '2', '5', '0');
```

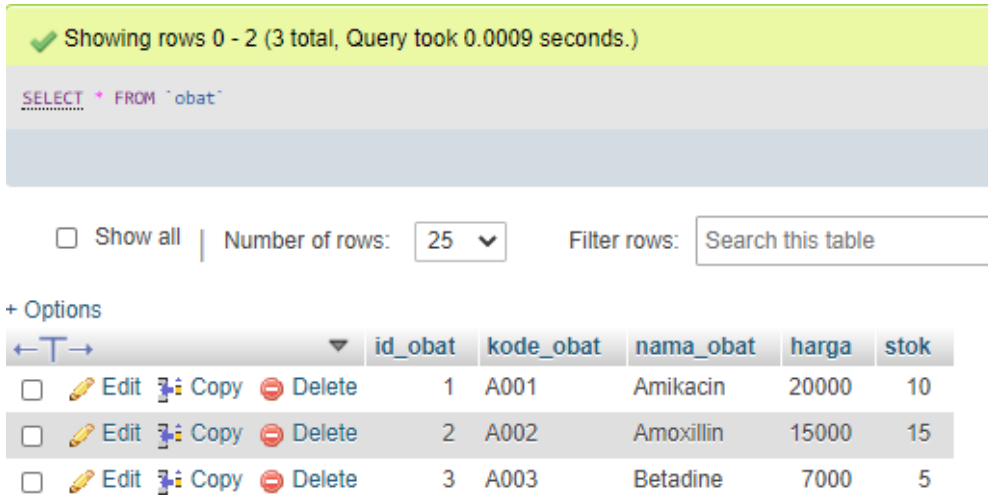
Gambar 33
Memasukkan Data Melalui SQL

Atau melalui halaman phpmyadmin seperti Gambar 34 berikut. (pilih salah satu cara saja melalui SQL atau melalui halaman phpmyadmin)



Gambar 34
Memasukan Data Melalui Phpmyadmin

Setelah data berhasil dimasukan maka Anda periksa kembali tabel obat. Maka kondisi tabel obat pada Amoxillin secara otomatis berkurang sebanyak jumlah yang dibeli pasien. Lihat Gambar 35 untuk melihat kondisi tabel obat.



Gambar 35
Kondisi Tabel Obat Setelah *Trigger Berjalan*

Pada Gambar 29 kondisi stok Amoxillin berjumlah 20. Namun, ada yang membeli obat tersebut sebanyak 5 pada jumlah di tabel transaksi_obat maka jumlah stok pada amoxillin berkurang secara otomatis berdasarkan jumlah data yang dimasukan pada tabel transaksi_obat.

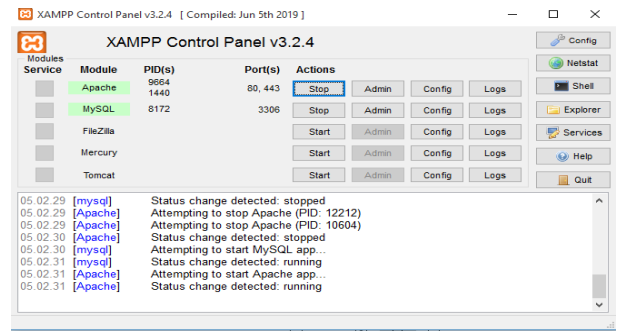
Jadi kesimpulan dari *trigger* adalah sebagai pemicu jika ada perubahan pada baris sebuah tabel di basis data. Perubahan bisa *insert*, *update*, atau *delete*. Silahkan Anda mencoba *trigger* pada kasus lainnya.

B.2. REPLIKASI

Setelah masing-masing komputer sudah saling terhubung, maka selanjutnya adalah bagaimana replikasi basis data dari komputer 1 (Windows 10) ke komputer 2 (Ubuntu). Anda akan melakukan konfigurasi pada masing-masing sistem operasi tersebut.

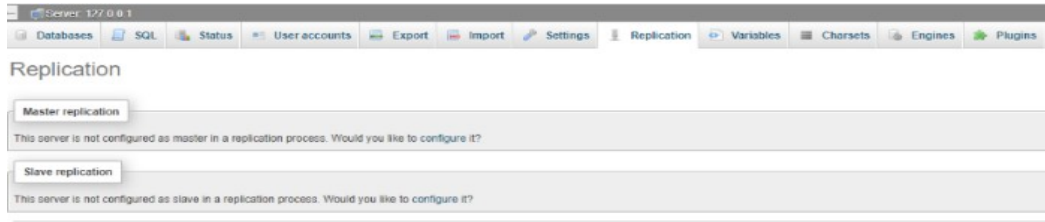
1. Konfigurasi pada Komputer 1 (Windows 10)

Pada konfigurasi windows 10 beberapa langkah yang dilakukan adalah aktifkan terlebih dahulu *services* pada control xampp. Jika belum aktif maka aktifkan seperti pada Gambar 36 berikut ini.



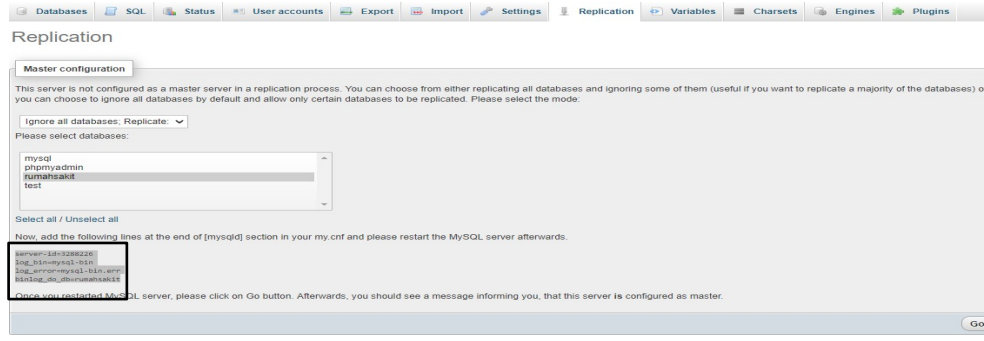
Gambar 36
Aktifkan services Apache dan MySQL pada XAMPP

Aktifkan *services* Apache dan MySQL dengan cara `klik Start` sampai dengan status warna hijau pada Apache dan MySQL seperti pada Gambar 9.108. setelah berhasil mengaktifkan Apache dan MySQL selanjutnya `buka browser` kemudian masuk ke halaman **http://localhost/phpmyadmin** dan `klik menu replication` seperti pada Gambar 37 berikut ini.



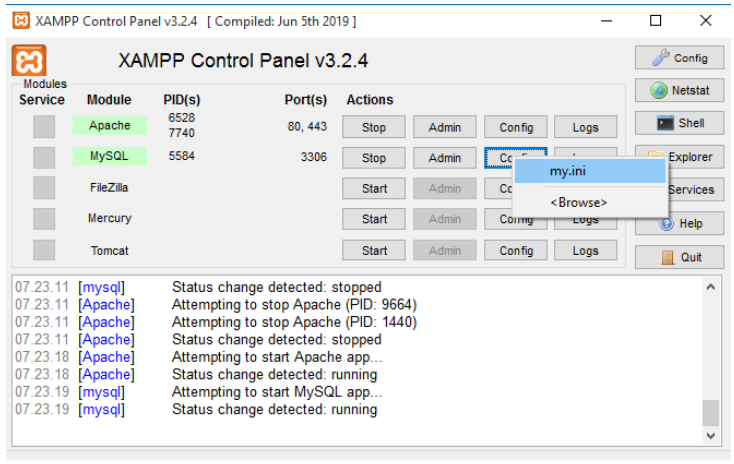
Gambar 37
Menu Replication

Pada menu ini, Anda akan melakukan konfigurasi server replikasi maka pada Master replication `klik configure it`. Kemudian `pilih basis data` yang akan direplikasi dalam praktikum ini menggunakan basis data yang telah dibuat pada Modul 9 kegiatan belajar 1 yaitu basis data **rumah sakit** seperti pada Gambar 38 berikut.



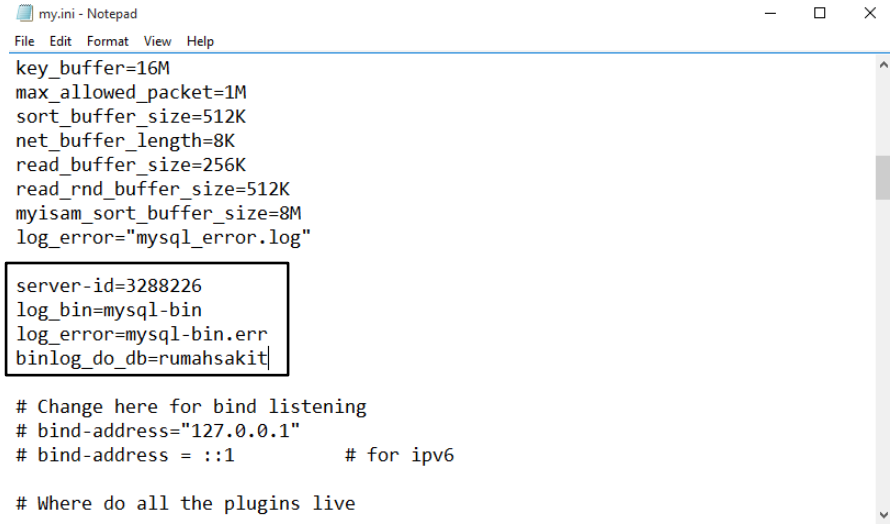
Gambar 38
Konfigurasi Master

Pada Gambar 38 terdapat 4 baris perintah yang harus Anda *copy* dan di-*paste* ke sebuah *file* bernama **my.ini**. Namun, perlu Anda perhatikan setelah melakukan *copy*, **jangan menekan terlebih dahulu tombol Go**. Lakukan terlebih *paste* perintah pada my.ini. Untuk mengakses *file* my.ini buka control xampp seperti pada Gambar 39 berikut ini.



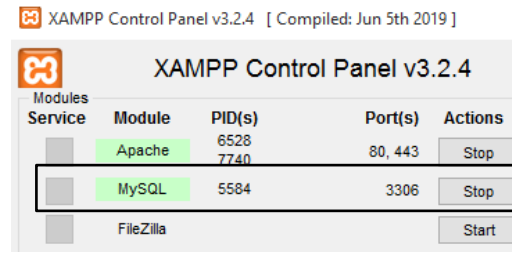
Gambar 39
Modifikasi file my.ini

Pada control xampp, `klik kanan pada tombol config` kemudian `klik my.ini` kemudian lakukan *paste* 4 baris perintah yang telah di-*copy* pada Gambar 38 seperti pada Gambar 40 berikut ini.



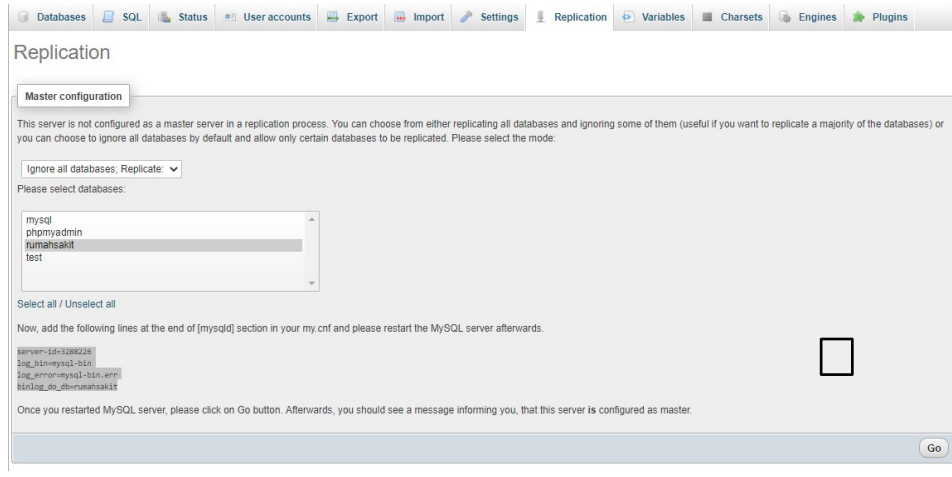
Gambar 40
Paste Perintah ke my.ini

Setelah berhasil di-*paste* ke *file* my.ini lakukan `klik save` kemudian `buka kembali control xampp` dan lakukan restart service dengan cara `klik tombol stop pada MySQL` kemudian `Start kembali`. Lihat Gambar 41 berikut ini



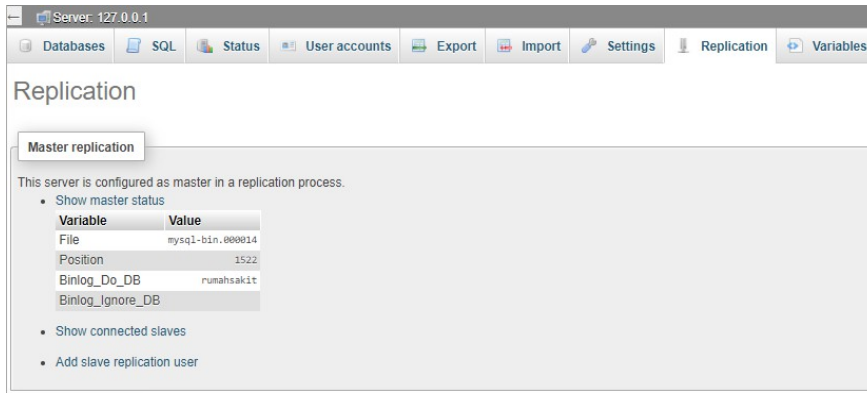
Gambar 41
Paste ke file my.ini

Setelah berhasil di-*restart service* MySQL kemudian lanjutkan kembali konfigurasi pada halaman replikasi seperti pada Gambar 42 `dan klik tombol GO`.



Gambar 42
Konfigurasi Replikasi dengan Tekan Tombol Go

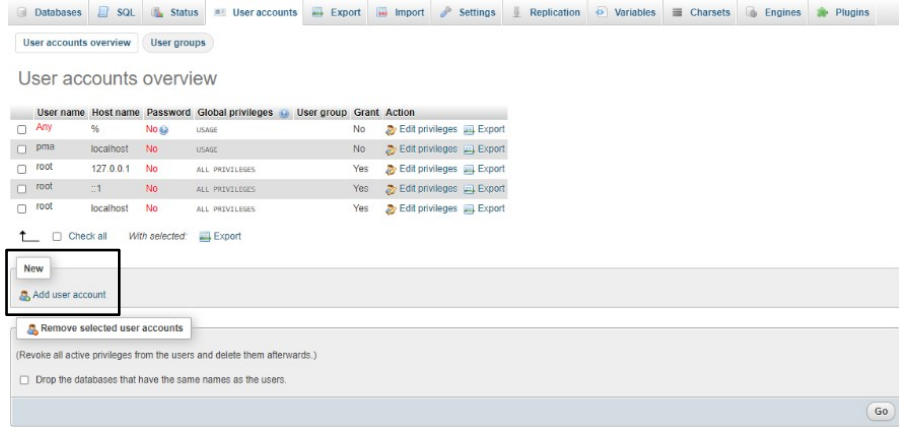
Jika selesai konfigurasi replikasi pada master maka selanjutnya lihat status replikasi masternya pada menu replikasi seperti pada Gambar 43 berikut ini.



Gambar 43
Status Replikasi Master

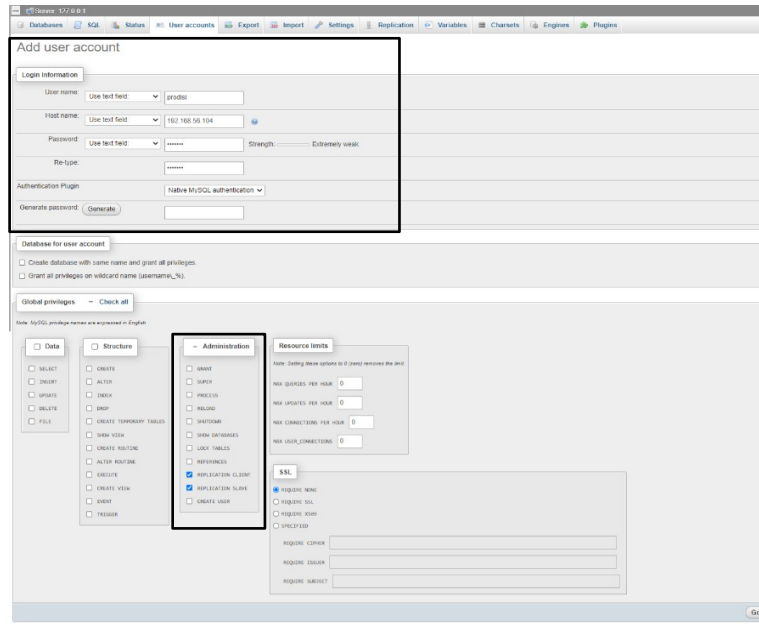
Jika berhasil maka akan tampil pada show master status yaitu variable dan value nya. Variable dan value ini akan dipakai pada saat konfigurasi pada komputer 2 (Ubuntu). Oleh karena itu, catat data master status tersebut.

Selanjutnya adalah membuat akun replikasi yang akan digunakan untuk mengakses basis data dari komputer 2. Untuk membuat akun tersebut `klik menu User Accounts`, kemudian `klik add user account` seperti pada Gambar 44 berikut ini.



Gambar 44
User Account

Setelah *add user account*, selanjutnya adalah menambahkan akun seperti pada Gambar 44 berikut ini.



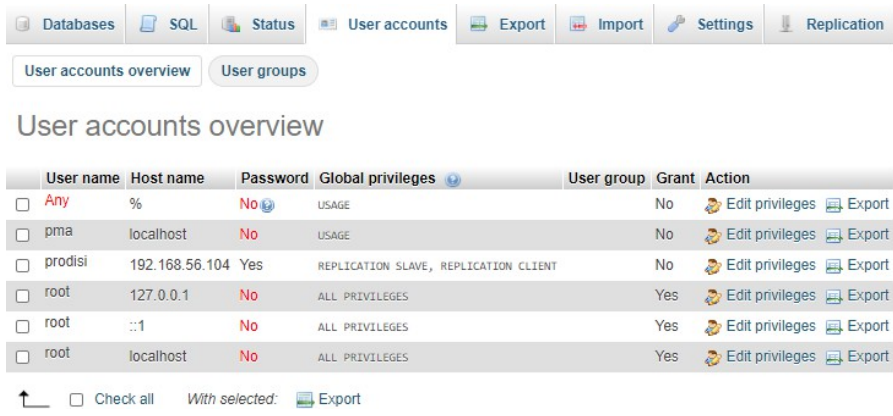
Gambar 45
Menambah Akun Replikasi

Pada Gambar 45 Anda tambahkan *username*, *hostname*, *password*, kemudian centang *replication client*, dan *replication slive*. Untuk lebih jelasnya perhatikan tabel 4 berikut ini.

Tabel 4
Penjelasan Akun Replikasi

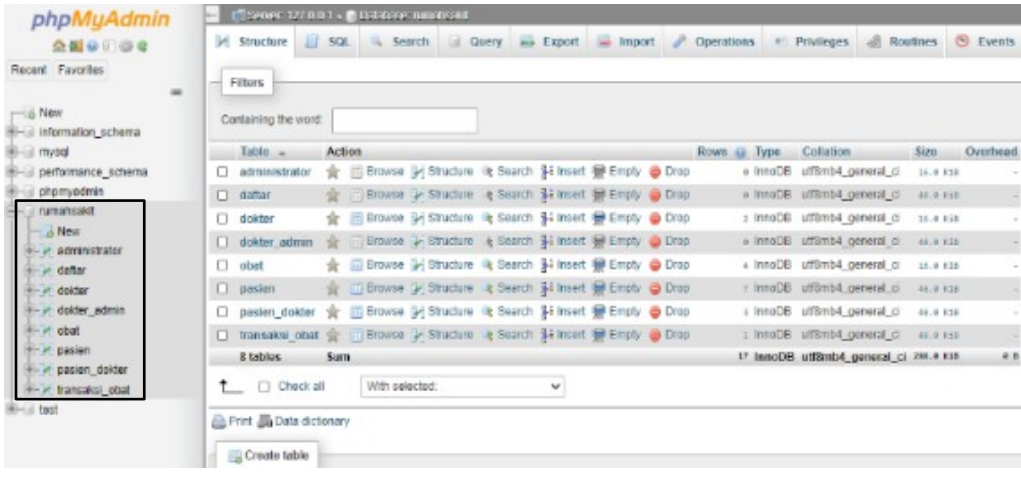
No	Menu	Keterangan
1	Username	Merupakan user account yang diperbolehkan dalam replikasi data
2	Hostname	Merupakan alamat komputer host / client yang akan menerima data replikasi. Alamat host disesuaikan dengan IP Address pada Ubuntu Anda
3	Password	Merupakan password yang diperbolehkan dalam replikasi data
4	Replication client	Mengaktifkan penggunaan pernyataan SHOW MASTER STATUS, SHOW SLAVE STATUS, dan SHOW BINARY LOGS kemudian memberikan hak istimewa ini ke akun yang digunakan oleh komputer client (Ubuntu) untuk menyambungkan ke server (Windows)
5	Replication slave	Memungkinkan akun untuk meminta pembaruan data pada server.

Perlu Anda ingat *hostname* isi dengan alamat ip address komputer 2 (Ubuntu) yang telah anda ketahui sebelumnya. Jika sudah lengkap selanjutnya `klik tombol Go`. Silahkan `klik menu User Accounts` untuk melihat hasilnya seperti Gambar 46 berikut.



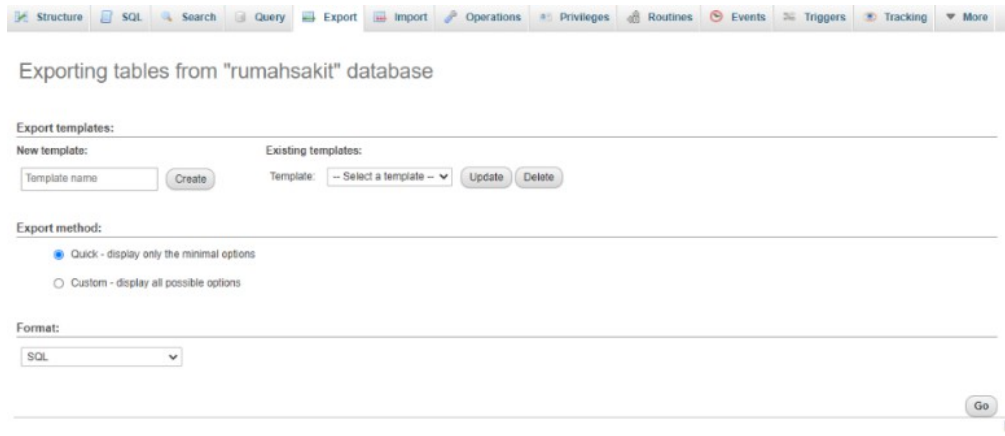
Gambar 46
Penambahan Akun Replikasi Selesai

Selanjutnya adalah melakukan *export* basis data rumah sakit dan simpan pada *flaskdisk* kemudian akan di-*import* pada basis data komputer 2 (Ubuntu). Langkah untuk melakukan *export* adalah pilih terlebih dahulu basis data rumah sakit dan akan tampil tabel-tabel pada basis data rumah sakit seperti pada Gambar 47 berikut ini.



Gambar 47
Pilih Basis Data Rumah Sakit

Selanjutnya `klik menu Export` dan akan tampil seperti pada Gambar 48 berikut ini.

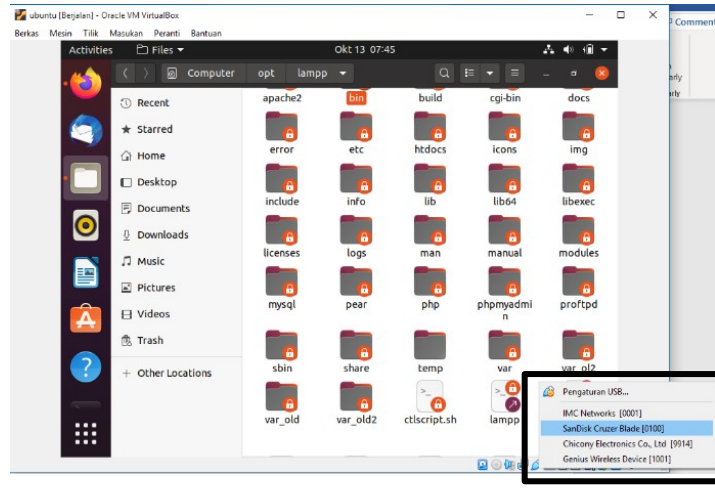


Gambar 48
Export Basis Data Rumah Sakit

`Tekan Go` dan akan menghasilkan file .sql kemudian simpan file tersebut dalam *flaskdisk* supaya dapat dipindahkan ke komputer 2 (Ubuntu). Proses konfigurasi pada komputer 1 selesai, selanjutnya akan dilakukan konfigurasi pada komputer 2 (Ubuntu).

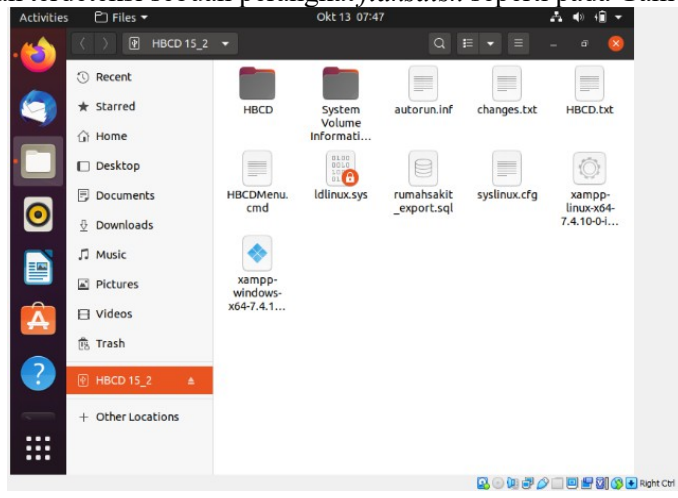
2. Konfigurasi Komputer 2 (Linux Ubuntu)

Pada konfigurasi komputer 2, langkah pertama Anda harus melakukan *import* basis data rumahsakit yang telah dilakukan pada Gambar 48. *File* tersebut telah disimpan dalam sebuah *flaskdisk* dan harus dipanggil ke dalam sistem operasi ubuntu. Cara untuk memanggil file tersebut adalah `klik kanan pada gambar flaskdisk` kemudian `klik perangkat flaskdisk` yang telah terdeteksi. Perhatikan Gambar 49 berikut ini.



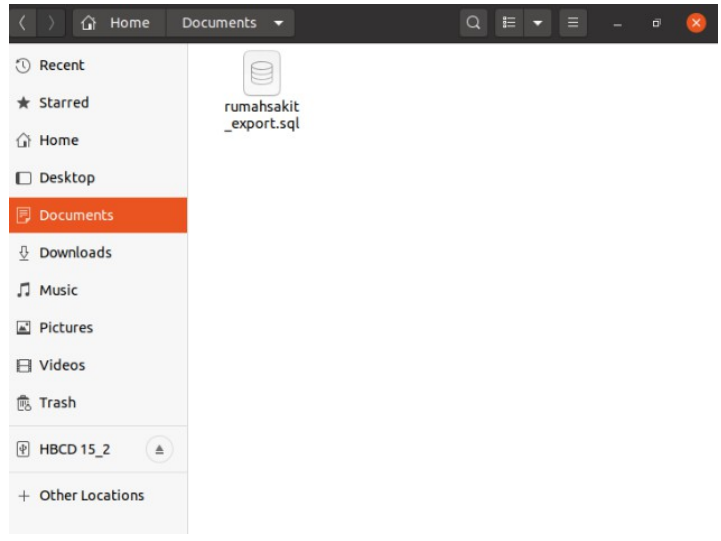
Gambar 49
Memanggil File dari *Flaskdisk*

Kemudian akan terdeteksi sebuah perangkat *flaksdisk* seperti pada Gambar 50 berikut.



Gambar 50
File dalam *Flaskdisk*

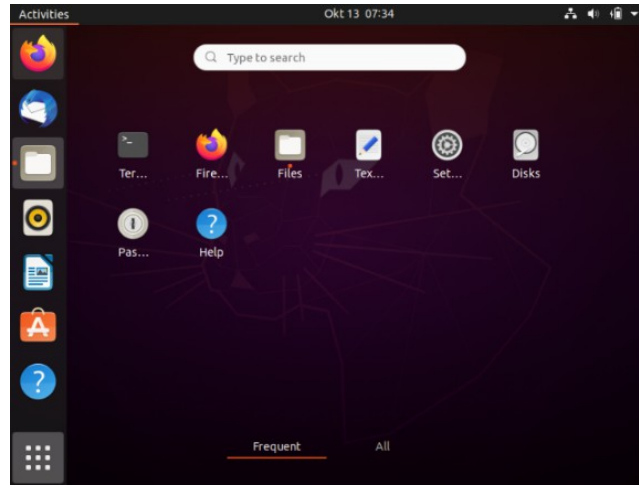
Gambar 50 sebagai contoh *file* **rumahsakit_export.sql** adalah *file* basis data rumahsakit yang akan di-*import* pada basis data MySQL dalam sistem operasi ubuntu. Oleh karena itu, Anda lakukan *copy file* rumahsakit dan silahkan *paste* misal pada *folder Documents* seperti pada Gambar 51 berikut ini.



Gambar 51
Paste File Rumah Sakit

Setelah *file* rumahsakit selesai disimpan pada sebuah *folder*, selanjutnya Anda harus melakukan *import* basis data rumahsakit tersebut dengan langkah awal adalah

jalankan xampp control dengan cara `buka terminal` terlebih dahulu seperti pada Gambar 52 berikut.

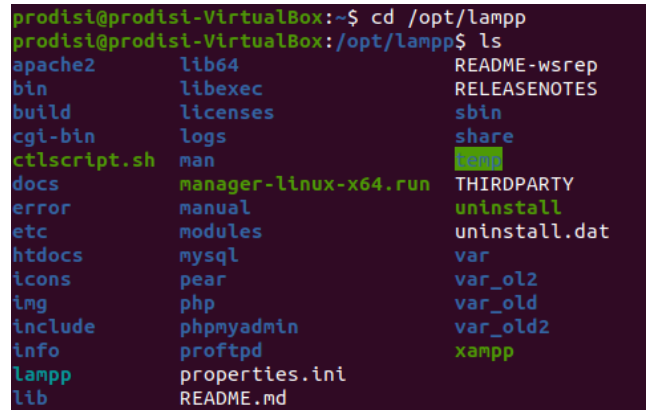


Gambar 52
Membuka Terminal

Selanjutnya masuk ke dalam direktori opt/lampp dengan cara ketik:

```
cd /opt/lampp
```

`tekan ENTER kemudian ketik ls (list)` dan akan tampil beberapa *file* yang ada di dalam *folder* lampp. Dalam *folder* lampp ada sebuah *file* yang bernama **manager-linux-xc64.run.** Perhatikan Gambar 53 berikut ini.

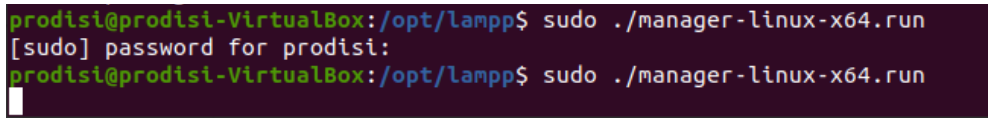


Gambar 53
List Direktori *Folder* Lampp

File tersebut merupakan *file control* xampp yang akan dijalankan. Untuk menjalankannya Anda ketikan perintah

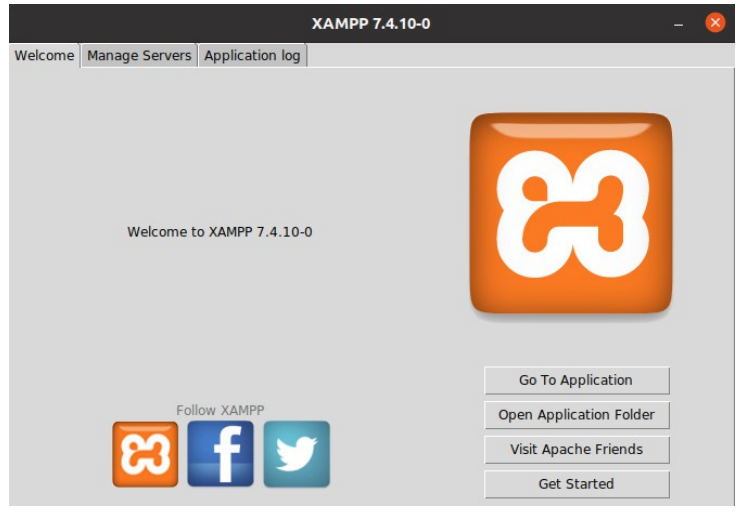
sudo ./ manager-linux-x64.run

kemudian akan meminta *password* dan isikan sesuai *password* akun yang telah Anda buat. Lihat Gambar 54 berikut ini.



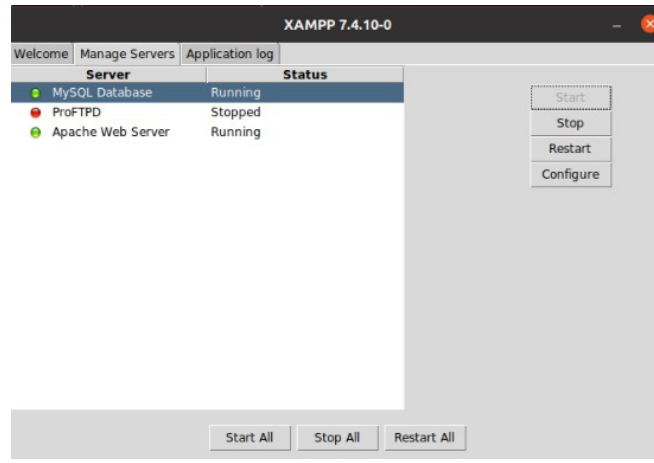
Gambar 54
Menjalankan File Manager-Linux-x64.run

Jika langkah tersebut berhasil maka akan menampilkan sebuah *control* xampp seperti pada Gambar 55 berikut ini.



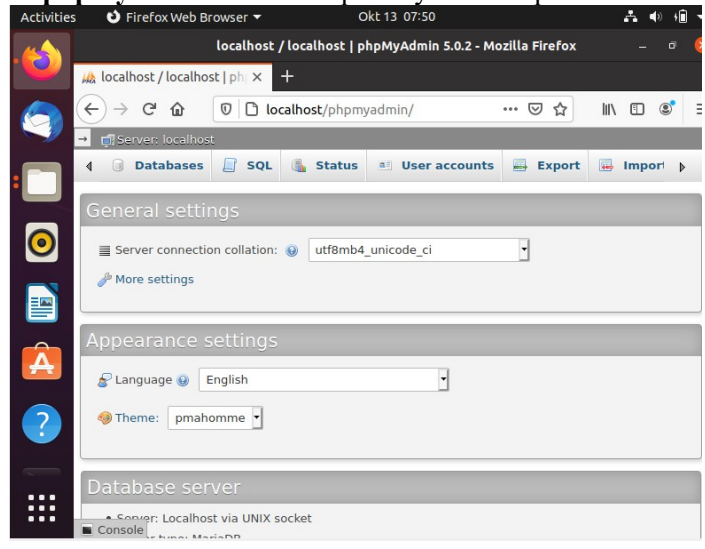
Gambar 55
Control XAMPP pada Ubuntu

Pada *control* xampp, `pilih menu tab Manage Servers` dan jalankan *service* dengan cara `klik tombol Start`.



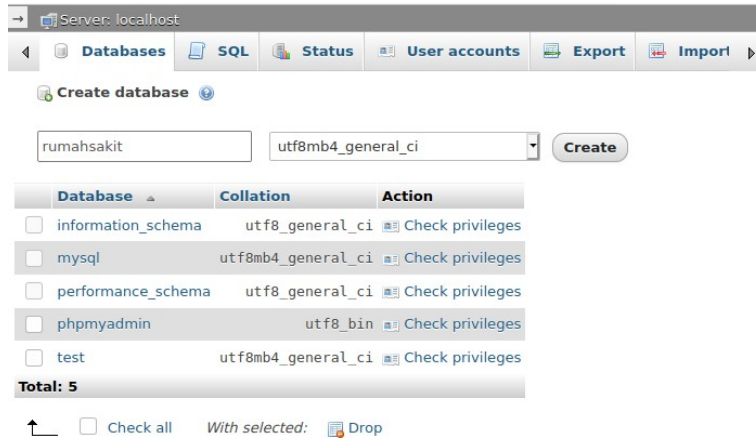
Gambar 56
Menjalankan *Service* Apache dan MySQL

Jika sudah berhasil dijalankan maka tandanya dapat dilihat pada status *Running*. Selanjutnya buka browser Mozilla dan masukan alamat pada URL **http://localhost/phpmyadmin**. Maka tampilannya akan seperti Gambar 57 berikut ini.



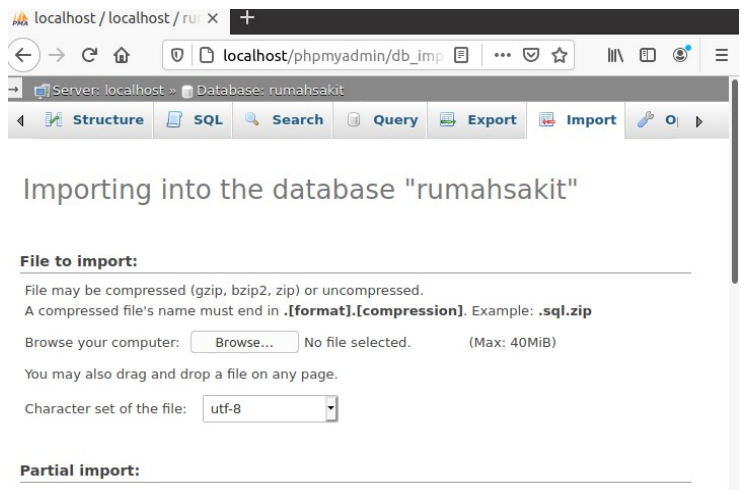
Gambar 57
Halaman Phpmyadmin

Jika sudah terbuka seperti Gambar 57, selanjutnya `klik menu Databases` dan buat nama basis data baru rumahsakit dan `klik Create` seperti pada Gambar 58 berikut ini.



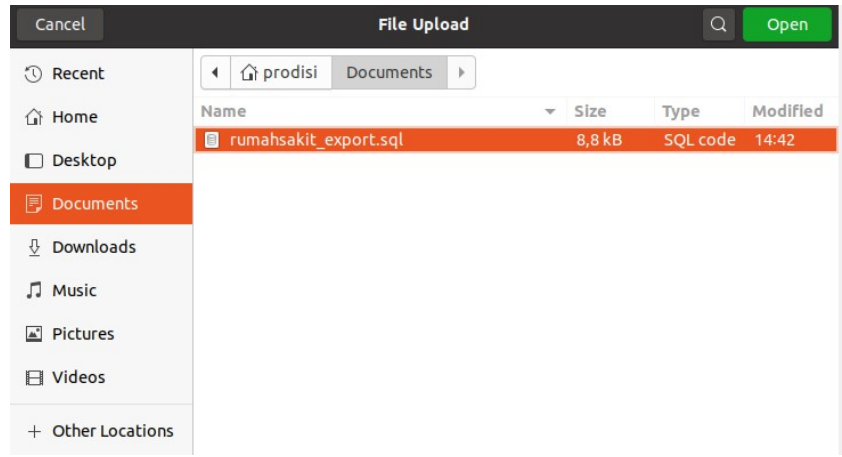
Gambar 58
Membuat Basis Data Rumah Sakit

Selanjutnya lakukan *import* dengan cara `klik menu Import` dan akan tampil tombol **Browse...** seperti pada Gambar 59 berikut ini.



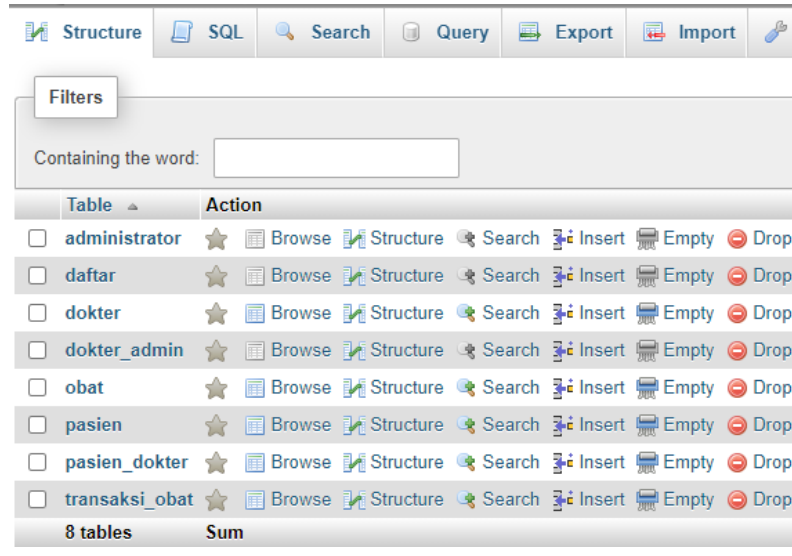
Gambar 59
Membuat Basis Data Rumah Sakit

Selanjutnya `klik tombol Browse` dan lakukan pemanggilan *file* rumah sakit yang telah disimpan pada *folder* Documents seperti pada Gambar 60 berikut.



Gambar 60
Memanggil File Rumah Sakit

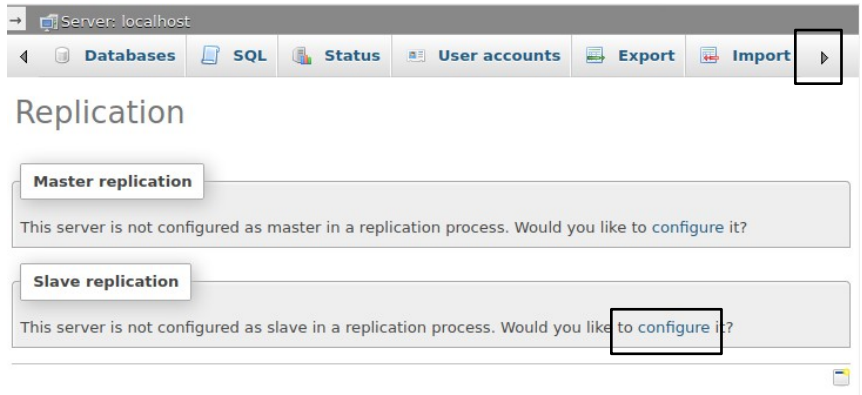
Jika sudah menemukan file rumah sakit yang telah dibuat pada Kegiatan Belajar 1, selanjutnya tekan pilih *file* tersebut dan `tekan tombol open` di pojok kanan atas. Selanjutnya silahkan klik tombol Go. Jika berhasil maka akan tampil tabel-tabel rumahsakit sama seperti pada komputer 1. Lihat Gambar 61 berikut ini.



Gambar 61
Tabel-tabel Rumah Sakit

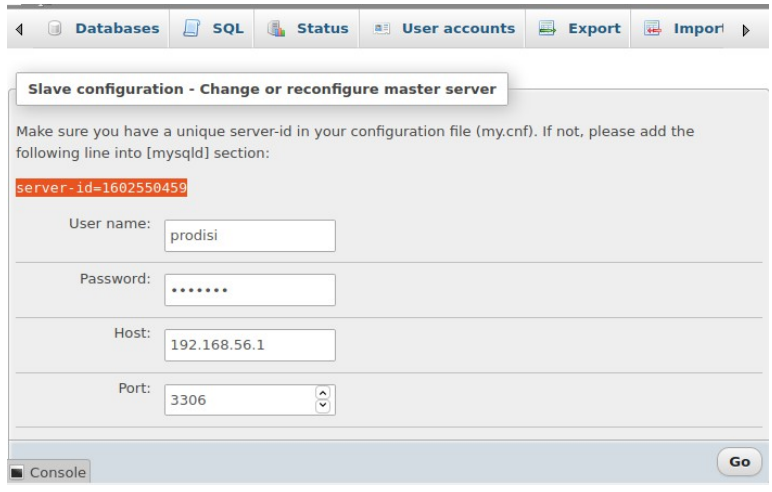
Proses *import* basis data rumahsakit telah selesai. Selanjutnya akan dilakukan konfigurasi replikasi pada komputer 2.

Pada praktikum ini akan dilakukan replikasi *master to slave* artinya komputer 2 hanya akan menerima perubahan basis data dari komputer 1 saja. Untuk melakukan konfigurasi replikasi basis data langkah `pertama klik menu Replication` di menu atas dan akan tampil seperti pada Gambar 62. Jika tidak tampil silahkan `klik tanda >.`



Gambar 62
Tampilan Menu Replikasi

Karena tahapan ini akan melakukan konfigurasi pada sisi *slave*, `maka klik configure it` pada *slave replication* dan akan tampil seperti pada Gambar 63 berikut ini.



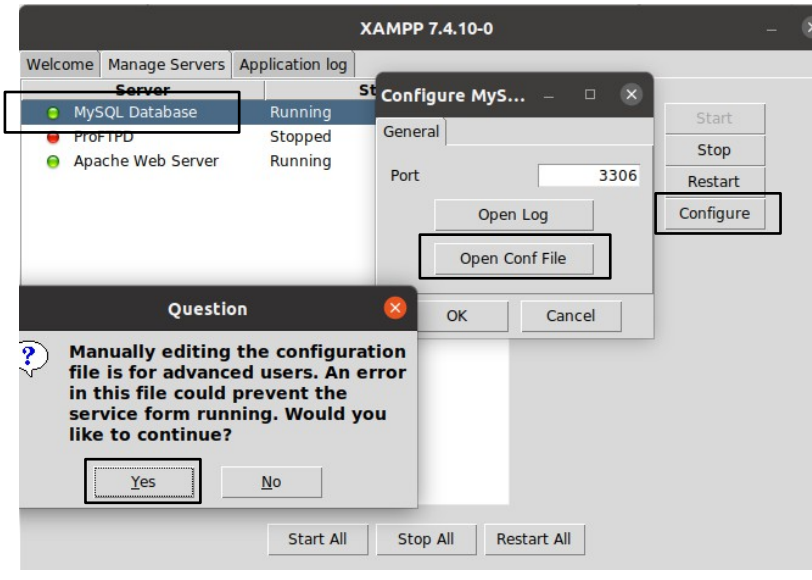
Gambar 63
Slave Configuration

Pada *slave configuration*, ada beberapa komponen yang harus diisi. Penjelasannya lihat 5 berikut ini.

Tabel 5
Konfigurasi Slave

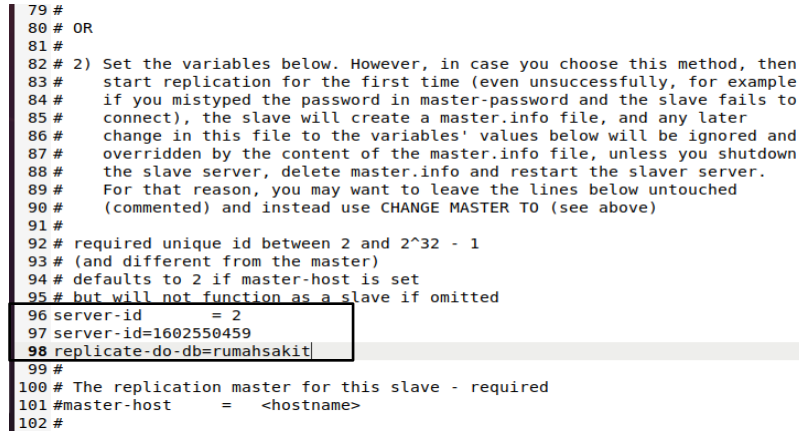
No	Konfigurasi	Penjelasan
1	Server-id	Server-id merupakan id server komputer 2 (Ubuntu).
2	User name	Isi dengan *username* yang telah dibuat pada komputer 1 (Windows 10)
3	Password	Isi dengan *password* yang telah dibuat pada komputer 1 (Windows 10)
4	Host	Isi dengan alamat ip address yang telah dibuat pada komputer 1 (Windows 10)
5	Port	Isi dengan port 3306

Perlu diingat, jika sudah konfigurasi diisi, **Anda jangan melakukan klik Go terlebih dahulu** namun dalam posisi tersebut lakukan *copy* server-id dan buka kembali control xampp untuk melakukan modifikasi pada *file* my.cnf seperti Gambar 64 berikut ini.



Gambar 64
Konfigurasi *file my.cnf*

Untuk membuka *file* my.cnf, pada control xampp `klik MySQL Databases` → `kemudian Configure` → `Open Conf File` → `Yes`. Maka akan terbuka *file* my.cnf seperti pada Gambar 65 berikut ini.



Gambar 65
Modifikasi *file my.cnf*

Pada file my.cnf, cari dan perhatikan server-id =2 jika ada tanda # diawal maka hilangkan tanda tersebut. Selanjutnya tambahkan server id yang sebelumnya ditampilkan pada Gambar 63 dan ditambah replicate-do-db=rumahsakit. Sehingga hasilnya seperti pada Gambar 65. setelah ditambahkan selanjutnya disimpan dengan cara `klik tombol save`. Biasanya ketika disimpan terjadi error karena hak akses *file* bersifat *read only*. Oleh karena itu, hak akses *file* harus diubah terlebih dahulu menjadi *read write*. Untuk mengubahnya Anda `buka terminal` kemudian masuk `ke folder /opt/lampp/etc` dengan cara mengetikan perintah:

```
cd /opt/lampp/etc
```

Kemudian ketikan ls untuk mengetahui file-file di dalam folder etc. Perintah untuk mengubah hak akses adalah dengan cara mengetikan perintah berikut.

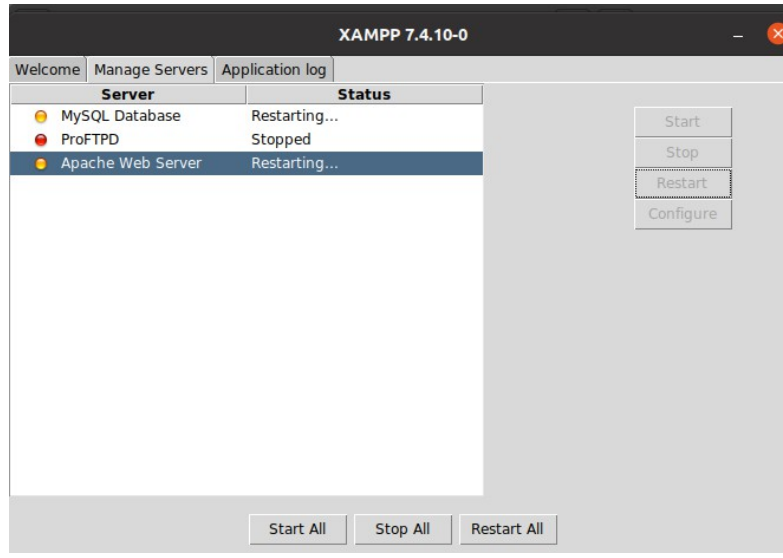
```
sudo chmod a+rwx my.cnf
```

kemudian masukan *password* akun ubuntu. Langkah-langkah tersebut dapat Anda lihat pada Gambar 66 berikut ini.



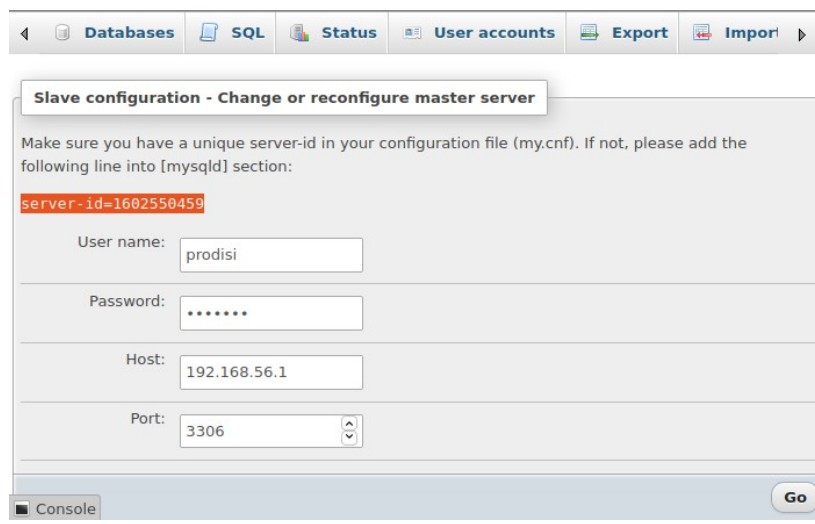
Gambar 66
Mengubah Hak Akses *file my.cnf*

Setelah berhasil mengubah hak akses *file* my.cnf klik save. Kemudian kembali ke *control* xampp dan lakukan *restart services* Apache dan MySQL dengan cara `klik tombol restart`. Lihat Gambar 67 berikut ini.



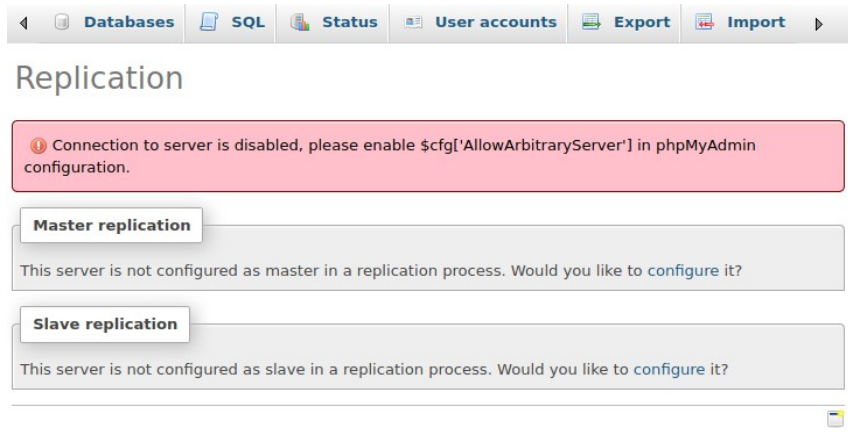
Gambar 67
Restart Services

Selanjutnya setelah services berhasil di-*restart*, kembali ke halaman replikasi dan `klik Go`. Seperti pada Gambar 68 berikut ini.



Gambar 68
Konfigurasi Slave Selesai

Selanjutnya jika setelah `ditekan Go` ada peringatan Connection server disable, please enable $cfg[‘AllowArbitraryServer’] in phpMyAdmin configuration seperti pada Gambar 69 berikut ini.



Gambar 69
Peringatan Server Disable

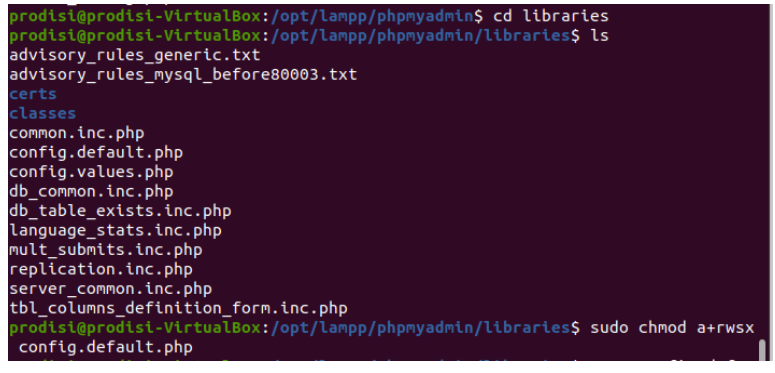
Maka selanjutnya Anda harus melakukan *enable* pada *file* config.default pada *folder* libraries. Namun, sebelum melakukan *edit* pada *file* tersebut Anda harus mengatur hak akses *file* config.default dengan cara buka terminal terlebih dahulu, kemudian masuk ke *folder* libraries dengan cara perintah:

```
cd /opt/lampp/phpmyadmin/libraries
```

kemudian atur hak akses menjadi read write dengan cara:

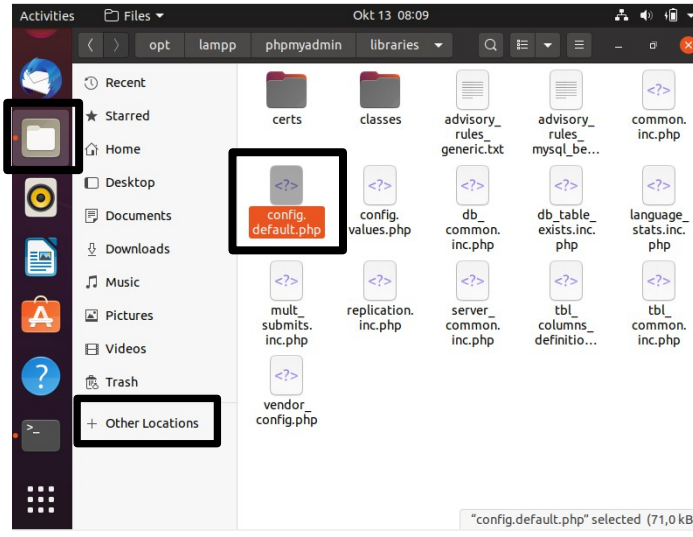
```
CHANGE.php
```

Untuk lebih jelasnya, lihat Gambar 70 berikut ini.



Gambar 70
Mengatur Hak Akses File config.default.php

Setelah hak akses diatur, selanjutnya Anda buka *folder* `/opt/lampp/phpmyadmin/libraries/config.default.php` pada `other location`. Lihat Gambar 71 berikut ini.



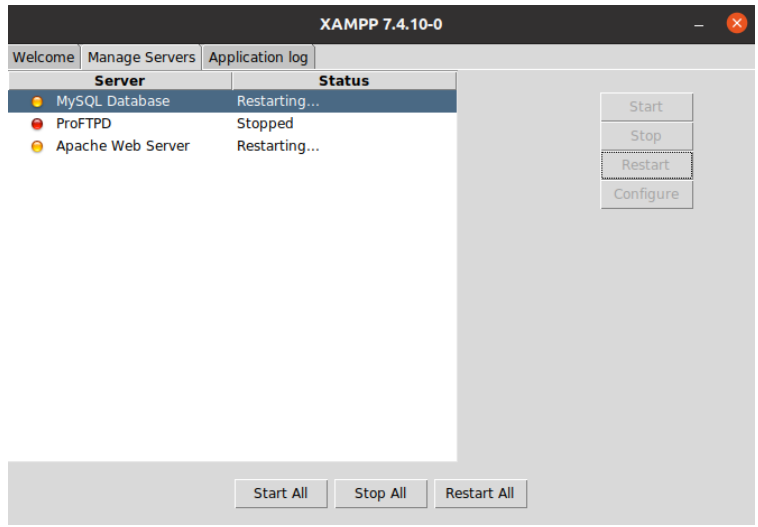
Gambar 71
Membuka file config.default.php

Lakukan modifikasi pada *file* config.default.php dengan mengubah **$cfg[‘AllowArbitraryServer’] = false** menjadi **true** seperti pada Gambar 72 berikut ini.



Gambar 72
Ubah Status AllowArbitraryServer menjadi True

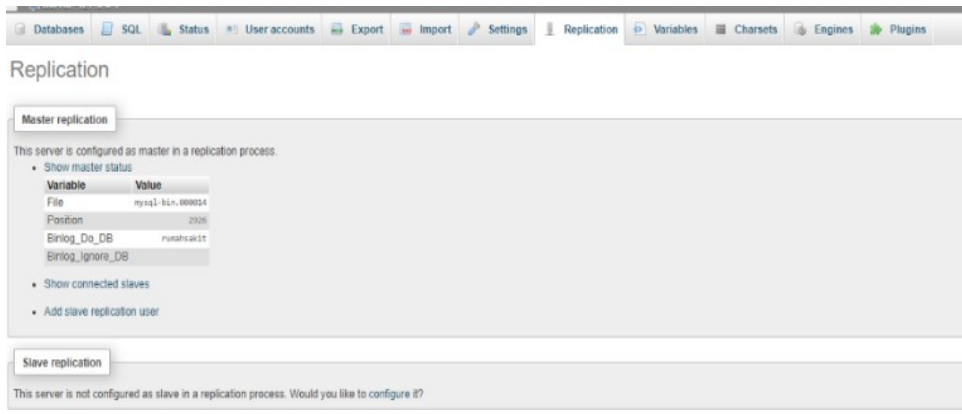
Jika selesai melakukan modifikasi pada *file* config.default.php *restart service* Apache dan MySQL pada *control* xampp. Seperti pada Gambar 73 berikut ini.



Gambar 73
Restart Services Apache dan MySQL

Jika tidak dilakukan *restart services* maka perubahan yang dilakukan pada *file* config.default.php tidak akan terjadi.

Langkah terakhir adalah melakukan master pada *slave*. Namun sebelum mengubah master Anda harus mengecek kembali status master pada komputer 1 (Windows 10) dengan cara `lihat halaman phpmyadmin` → `menu replication` → `show master status`. Lihat Gambar 74 berikut ini.



Gambar 74
Status Master pada Komputer 1 (Windows 10)

Catatlah status master tersebut karena akan digunakan untuk melakukan perubahan master pada komputer 2 (Ubuntu). Setelah dicatat buka kembali terminal

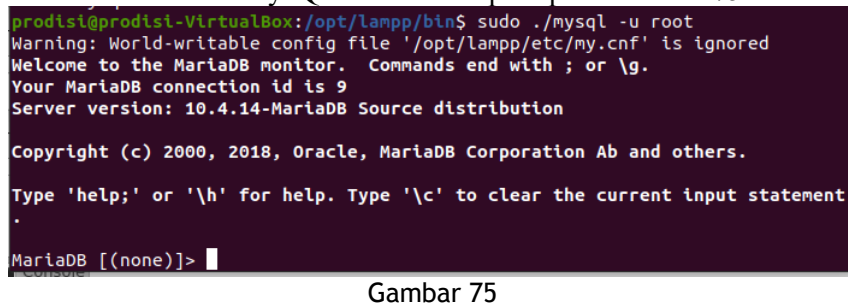
pada komputer 2 dan `masuk ke folder /opt/lampp/bin` dengan cara mengetikan perintah:

```
cd /opt/lampp/bin
```

kemudian masuk ke mysql dengan cara mengetikan perintah

```
sudo ./mysql -u root
```

maka Anda akan masuk ke MySQL/MariaDB seperti pada Gambar 75 berikut ini.

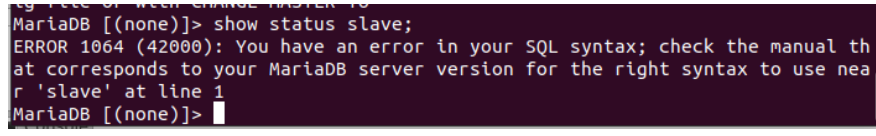


Gambar 75
Masuk ke MariaDB

Setelah berhasil masuk ke mariaDB, Anda lakukan cek terlebih dahulu *slave* dengan cara mengetikan perintah:

```
show status slave;
```

Hasilnya dapat Anda lihat pada Gambar 76 berikut ini.



Gambar 76
Cek Status Slave

Hasilnya adalah *error* artinya *slave* dalam keadaan **tidak aktif** dan konfigurasi dapat terus dilanjutkan (konfigurasi dapat dilakukan jika *slave* dalam keadaan tidak aktif). Namun, jika slave dalam keadaan aktif maka harus di-*nonaktifkan* terlebih dahulu dengan cara mengetikan `stop slave;`

Jika *slave* sudah dalam keaadan tidak aktif, maka selanjutnya lakukan perubahan pada *master slave* dengan cara mengetikan perintah:

```
CHANGE MASTER TO
MASTER_HOST='192.168.56.1',
MASTER_USER='prodisi',
MASTER_PASSWORD='prodisi',
MASTER_LOG_FILE='mysql-bin.000014',
MATER_LOG_POS=2926;
```

Penjelasan dari perintah di atas dapat dilihat pada Tabel 6 berikut ini.

Tabel 6
Konfigurasi Slave

No	Konfigurasi	Penjelasan
1	CHANGE MASTER TO	Perintah awal mengubah master slave
2	MASTER_HOST='192.168.56.1',	Isi dengan alamat ip adress komputer 1 (Windows 10)
3	MASTER_USER='prodisi',	Isi dengan master *user* yang telah dibuat pada komputer 1 (Windows 10)
4	MATER_PASSWORD='prodisi',	Isi dengan master *password* yang telah dibuat pada komputer 1 (Windows 10)
5	MASTER_LOG_FILE='mysql-bin.000014',	-
6	MATER_LOG_POS=2926;	-

Untuk lebih jelasnya lihat Gambar 77 berikut ini.

```
MariaDB [(none)]> CHANGE MASTER TO
    -> MASTER_HOST='192.168.56.1',
    -> MASTER_USER='prodisi',
    -> MASTER_PASSWORD='prodisi',
    -> MASTER_LOG_FILE='mysql-bin.000014',
    -> MASTER_LOG_POS=2926;
Query OK, 0 rows affected (0.073 sec)

MariaDB [(none)]>
```

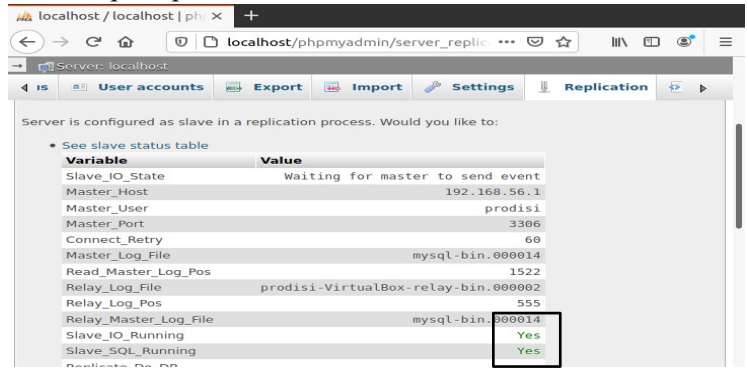
Gambar 77
Merubah *Master Slave*

Jika sudah diketikan perintah `tekan enter` dan akan tampil seperti Gambar 77. Selanjutnya adalah memulai *slave* dengan cara mengetikan `start slave` seperti pada Gambar 78 berikut ini.

```
MariaDB [(none)]> start slave;
Query OK, 0 rows affected (0.002 sec)
```

Gambar 78
Start Slave

Pada Gambar 78 *slave* telah berhasil dijalankan. Maka selanjutnya cek status *slave* pada komputer 2 (Ubuntu). Untuk melakukan pengecekan, kembali pada browser mozilla, pada halaman phpmyadmin `klik menu replication` dan `lihat status slave` seperti pada Gambar 79 berikut ini.

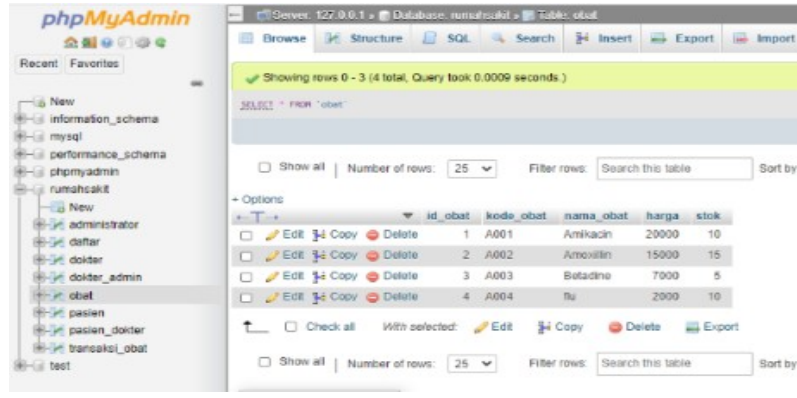


Gambar 79
Status slave

Perhatikan *slave_IO_Running* dan *Slave_SQL_Running*. Jika status keduanya **Yes,** maka konfigurasi *slave* berhasil artinya basis data pada komputer 2 siap menerima replikasi dari komputer 1. Namun, jika ada status yang No berarti konfigurasi belum berhasil dan Anda cek kembali langkah demi langkah sampai statusnya Yes.

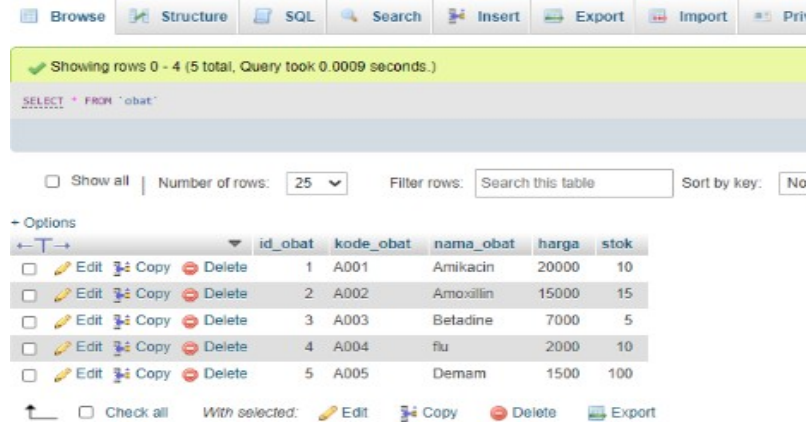
PENGUJIAN REPLIKASI

Tahapan selanjutnya adalah pengujian replikasi basis data. Untuk melakukan pengujian apakah replikasi telah berjalan dengan baik, maka lakukan perubahan basis data pada komputer 1 (Windows 10) kemudian lihat hasilnya pada basis data komputer 2 (Ubuntu). Langkah-langkah pengujian pertama-tama `buka tabel obat` pada komputer 1 seperti pada Gambar 80 berikut ini.



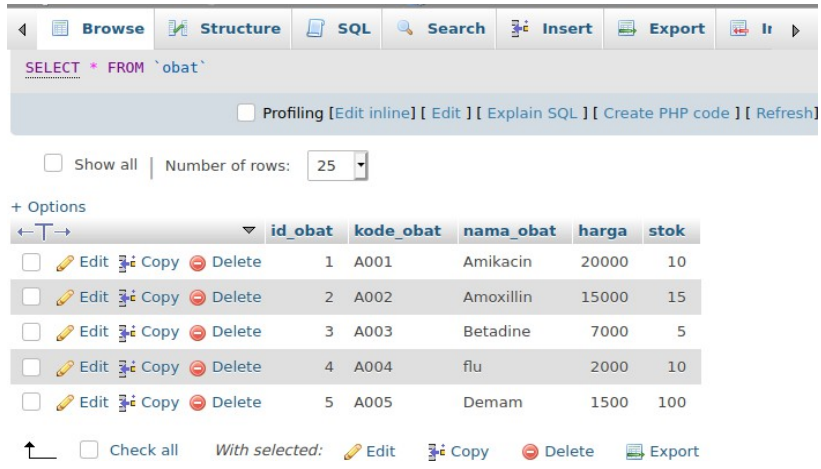
Gambar 80
Tabel Obat pada Komputer 1

Kemudian lakukan perubahan data (*insert/edit/delete*). Sebagai contoh menambah data pada tabel obat yaitu obat Demam dengan kode A005, harga 150, dan stok 100. Lihat Gambar 81 berikut ini.



Gambar 81
Hasil Penambahan Data pada Tabel Obat

Setelah dilakukan penambahan berikutnya adalah lakukan pengecekan pada tabel obat di komputer 2 (Ubuntu). Maka hasilnya akan secara otomatis bertambah percis sama dengan komputer 1. Lihat Gambar 82 berikut ini.



Gambar 82
Hasil Replikasi Data pada Komputer 2

Jika hasilnya sama seperti komputer 1, maka proses replikasi basis data telah berhasil dilakukan. Namun, jika belum berhasil maka silahkan ulangi langkah-langkah konfigurasi replikasi sampai berhasil.

BAB 3

Pelaporan dan Penilaian Praktik/Praktikum

Setelah melakukan praktikum *Praktikum Transaksi dan Basis Data Terdistribusi*, mahasiswa wajib menyusun laporan sebagai dasar penilaian.

TAHAP PELAPORAN DAN PENILAIAN

Laporkan kegiatan praktik yang telah dilakukan dengan mengupload link rekaman video yang menampilan:

a. Perkenalan identitas mahasiswa
b. Langkah-langkah praktik yang Anda lakukan disertai aktivitas kegiatan mahasiswa
c. Hasil praktik yang telah dilakukan
d. Kata-kata penutup dari mahasiswa